Room 1: Passive Recon

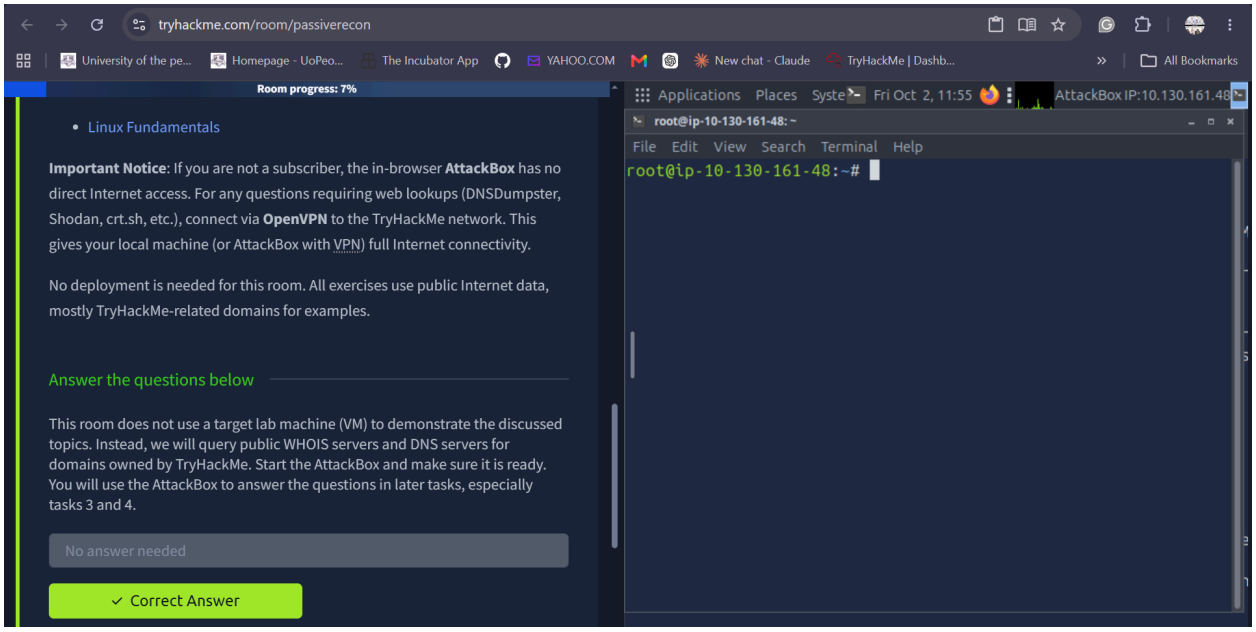
Platform: TryHackMe
Objective: Learn to gather information about a target from public sources without contacting it, using WHOIS, DNS lookups, DNSDumpster, certificate transparency logs, and Shodan.

Task 1: Introduction

Notes:

> This room introduces passive reconnaissance: collecting intelligence from public sources without contacting the target. By the end, I should be able to use whois, dig, and nslookup, find subdomains with DNSDumpster and crt.sh, and gather information on exposed services with Shodan. No machine deployment is needed, because everything uses public internet data.

Screenshot:



Task 2: Passive Versus Active Recon

Notes:

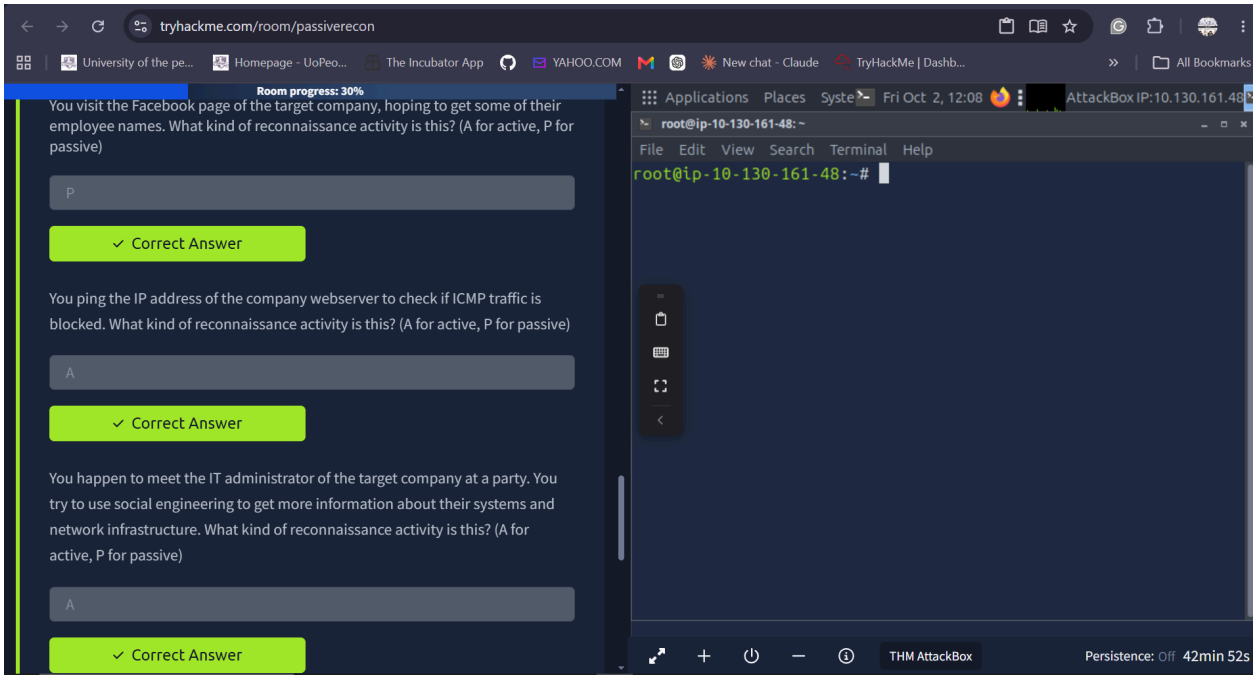
> **Reconnaissance** is the first phase of an attack or a penetration test, and it comes in two types.

- **Passive recon** uses only public information. No packets are sent to the target, so it can't be detected. Examples: public DNS records, certificate transparency logs, job postings, news, Shodan/Censys, and public GitHub repositories.
- **Active recon** means interacting with the target directly. It can be logged, detected, or blocked, and without authorisation it can be illegal. Examples: ping sweeps, port scanning (Nmap), directory brute-forcing, social engineering, and physical approaches.

Talking to an employee about their tech stack also counts as active recon, even though no packets are sent.

Defenders can monitor their own public footprint with Shodan/Censys alerts and certificate log watchers.

Screenshot:



Task 3: Whois

Notes:

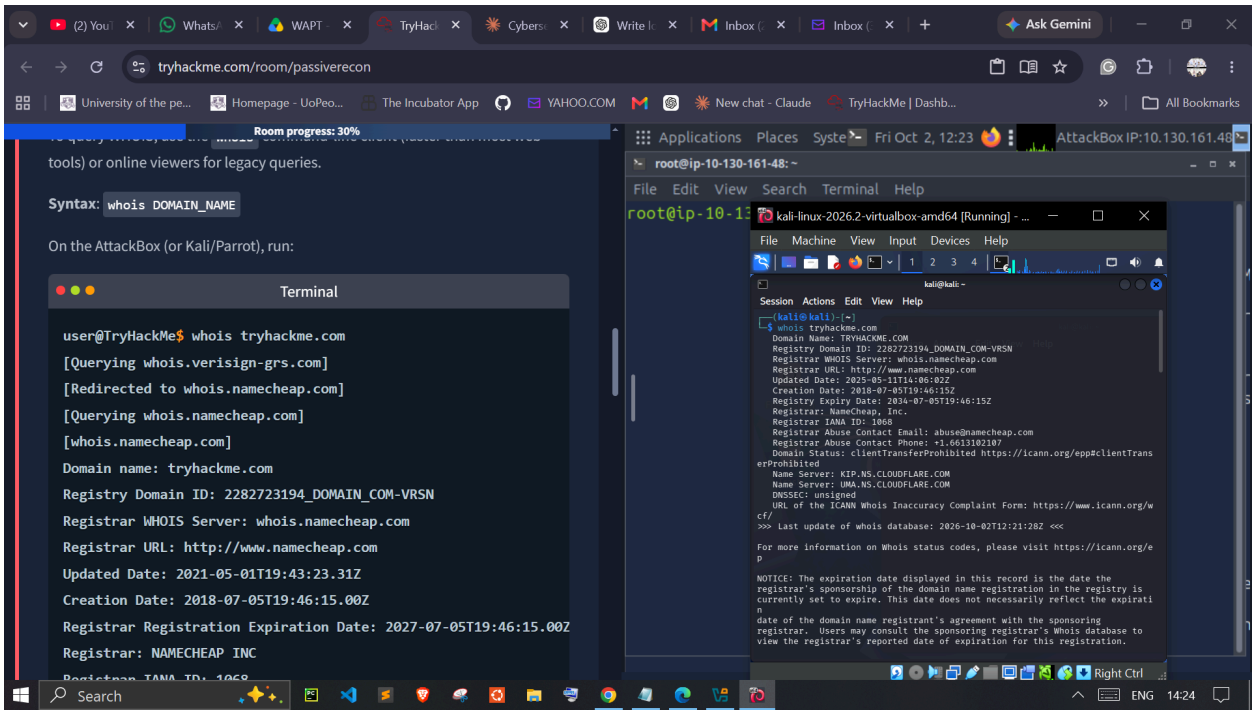
WHOIS is a protocol (TCP port 43) that returns domain registration details: the **registrar**, **registrant contact** (usually redacted now for privacy), **dates** (creation, updated, expiry), **name servers**, **status codes** (for example

clientTransferProhibited), and **abuse contacts**. WHOIS is being replaced by **RDAP**, which uses HTTPS and returns structured JSON.

Command used:

whois tryhackme.com

What the output showed:



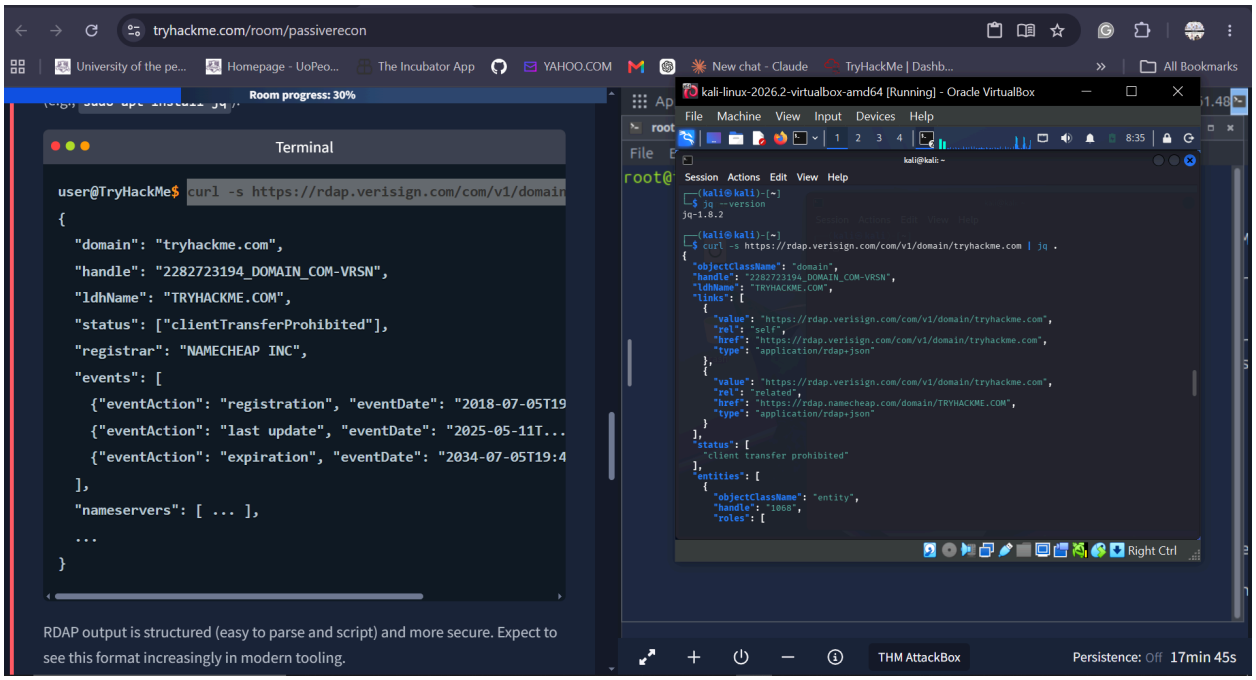
Field	Value
Registrar	NameCheap, Inc.
Creation Date	2018-07-05
Updated Date	2025-05-11
Registry Expiry Date	2034-07-05
Name Servers	KIP.NS.CLOUDFLARE.COM, UMA.NS.CLOUDFLARE.COM
Domain Status	clientTransferProhibited

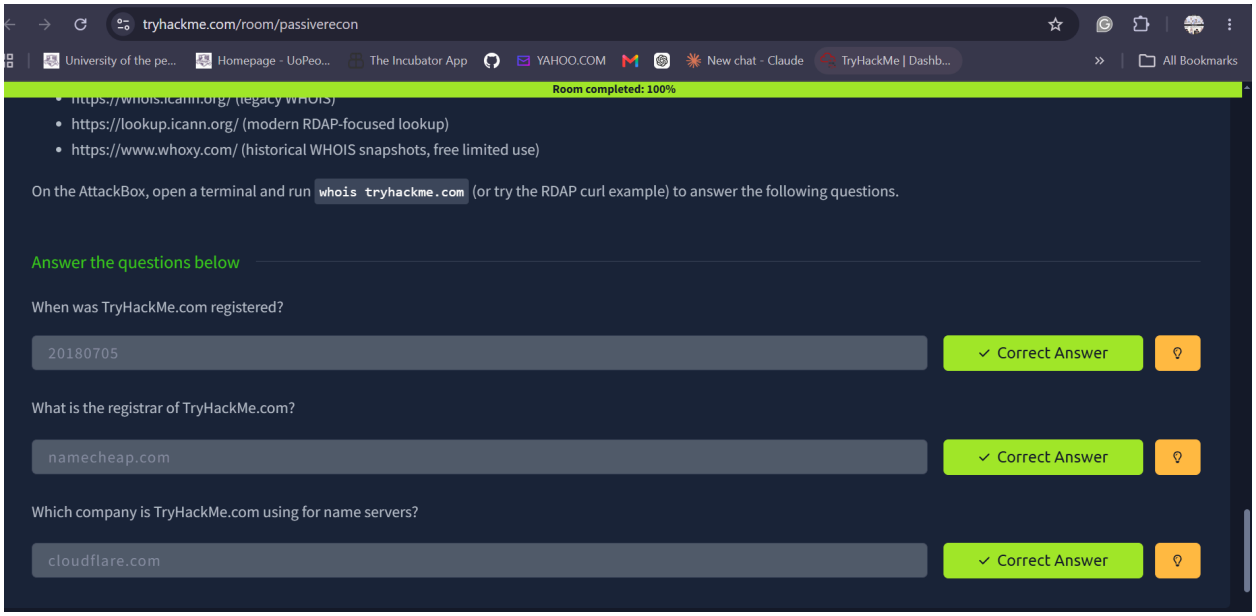
DNSSEC	unsigned

Problem-solving note:

whois first failed with "network is unreachable", and ping 8.8.8.8 showed 100% packet loss on the TryHackMe AttackBox. The AttackBox restricts general internet access, so I ran the lookup from a Kali VM that had internet access instead. I also hit a wrong-format answer on the registration date, so I matched the format to the answer box.

Screenshots:





Task 4: nslookup and dig

Notes:

nslookup and dig query DNS records without touching the target's servers, so they stay passive. dig is the preferred tool because its output is cleaner and it shows TTL values.

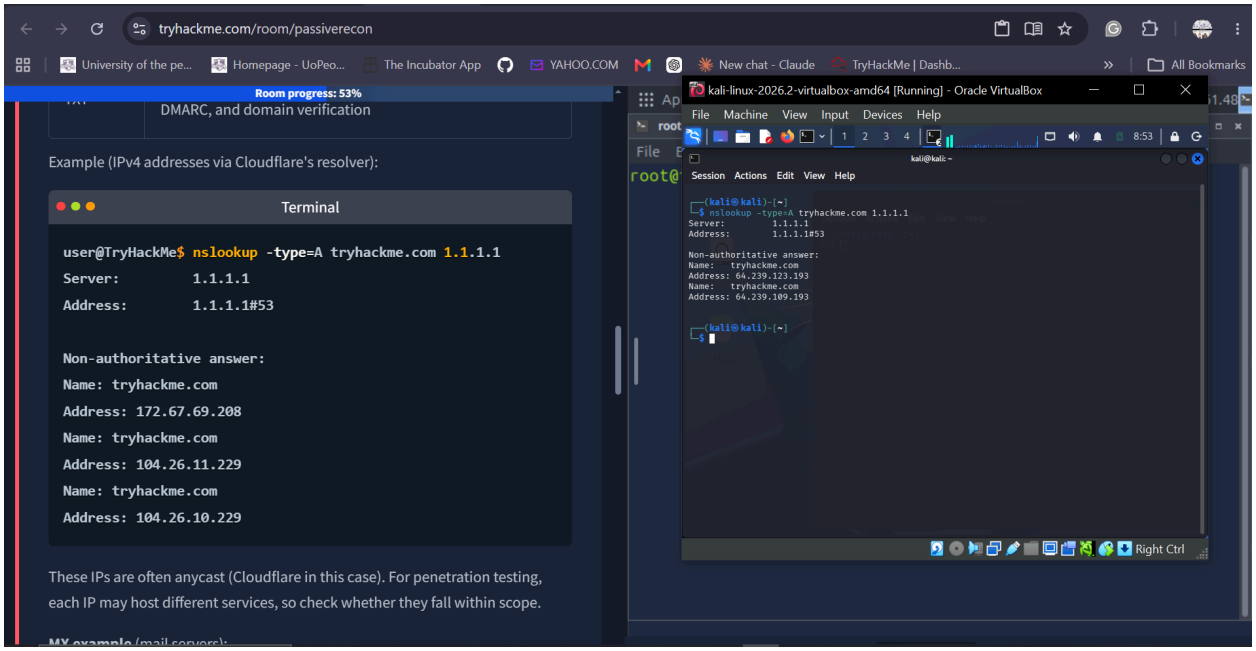
Record	Meaning
A	IPv4 address
AAAA	IPv6 address
CNAME	Alias pointing to another name
MX	Mail servers (lower number = higher priority)
SOA	Primary name server, admin email, zone serial
TXT	Text data such as SPF, DKIM, DMARC, verification strings

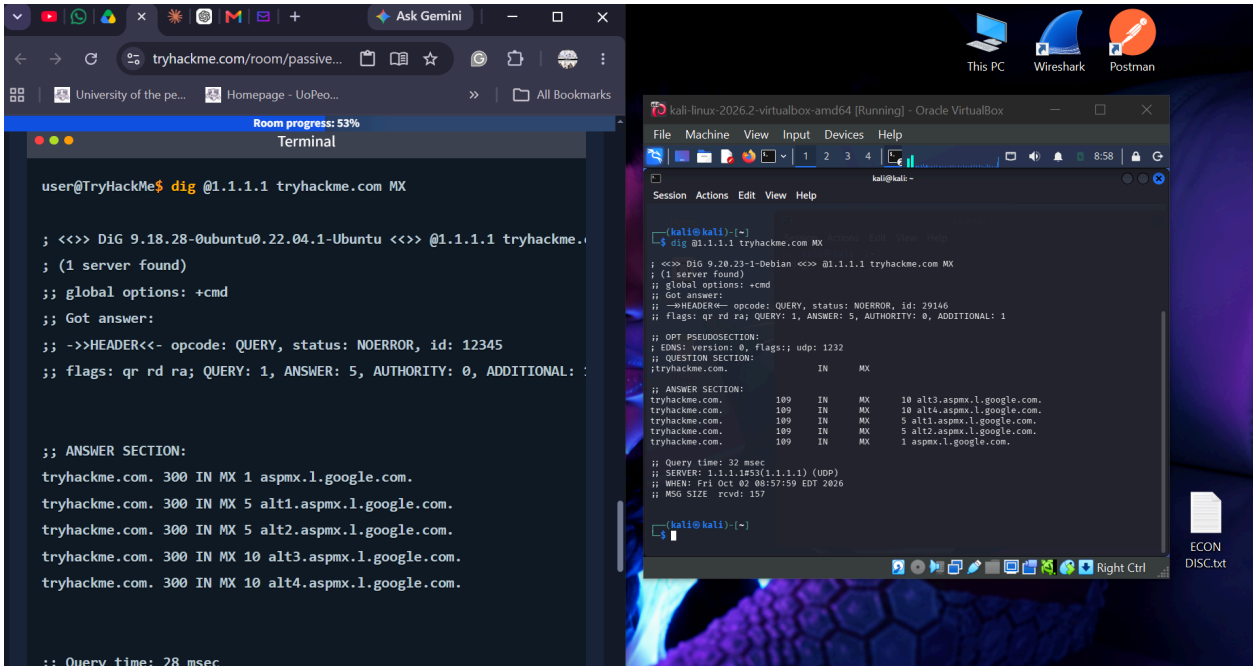
tryhackme.com's MX records point to Google (aspmx.l.google.com), so email is handled by Google Workspace. Its IPs are Cloudflare anycast addresses.

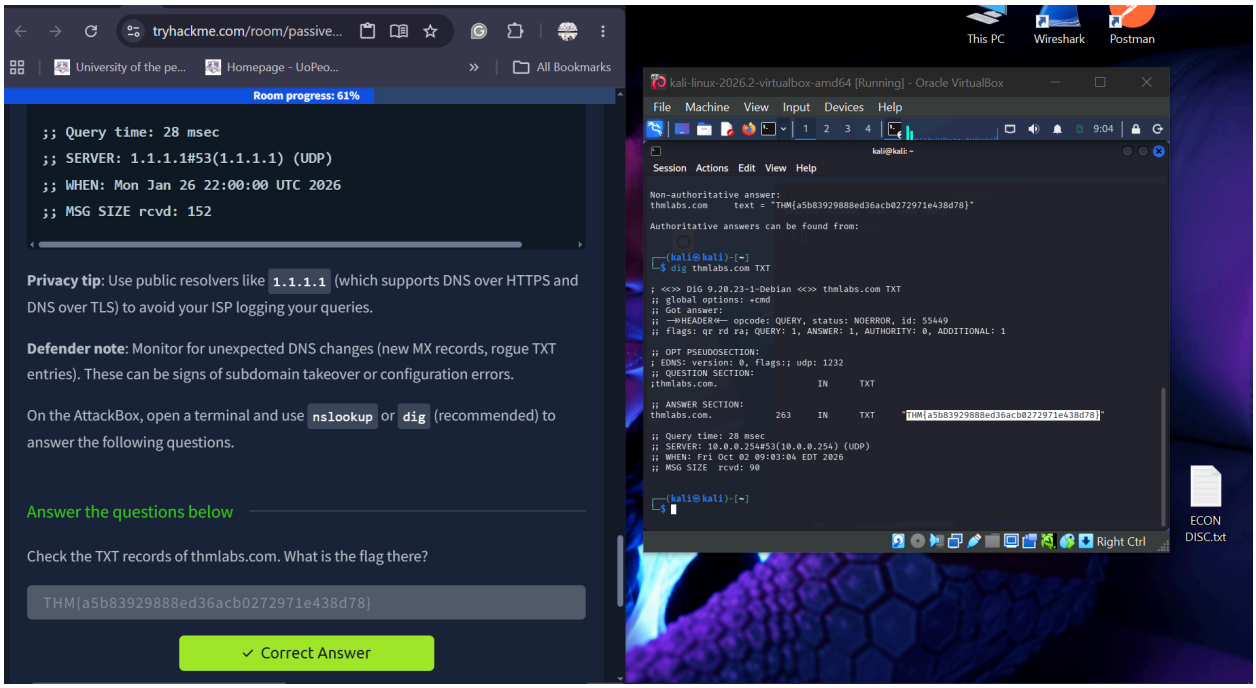
Commands to run and screenshot

nslookup -type=A tryhackme.com 1.1.1.1
nslookup -type=MX tryhackme.com
dig @1.1.1.1 tryhackme.com MX
dig tryhackme.com TXT

Screenshots:







Task 5: DNSDumpster and Certificate Transparency

Notes:

Normal DNS lookups only resolve names I already know. **DNSDumpster** collects public DNS data to reveal subdomains, IPs, MX, TXT, and CNAME records, and it draws a map of how they relate. It's passive because it doesn't brute-force anything.

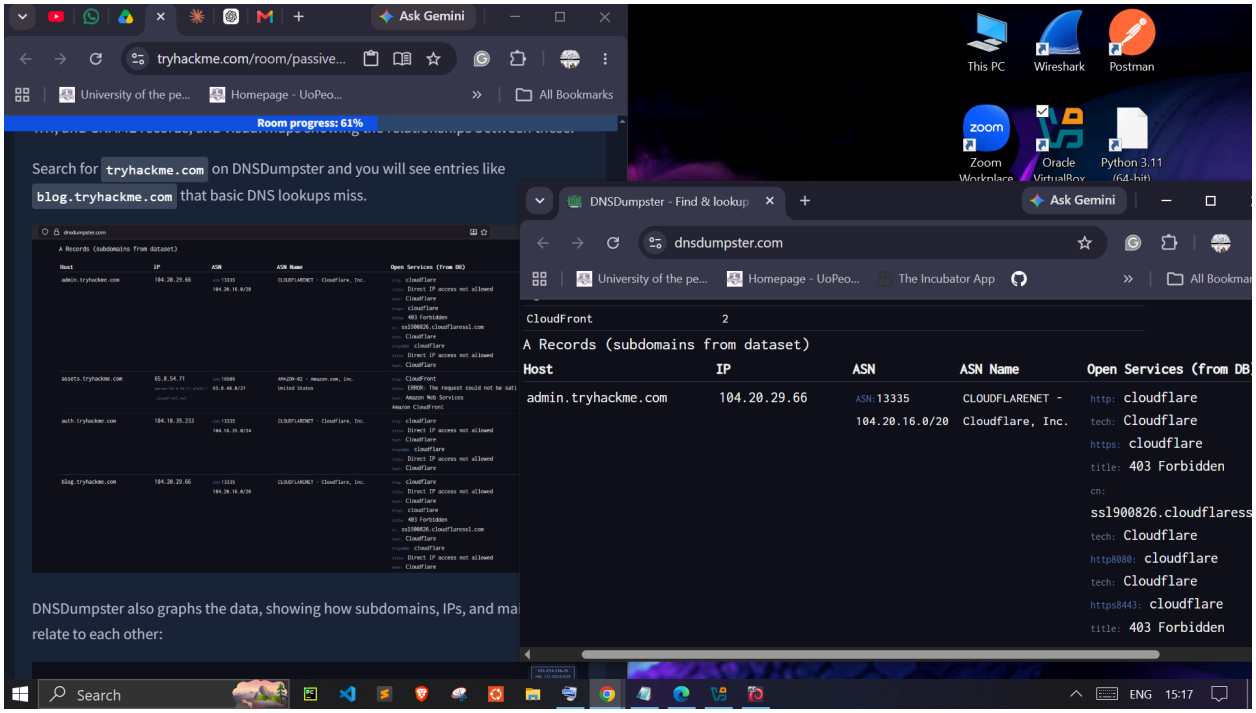
Certificate Transparency logs (searched at crt.sh with %.tryhackme.com) record every SSL/TLS certificate issued, and each certificate lists the domains and subdomains it covers. This is often the most effective passive way to find subdomains.

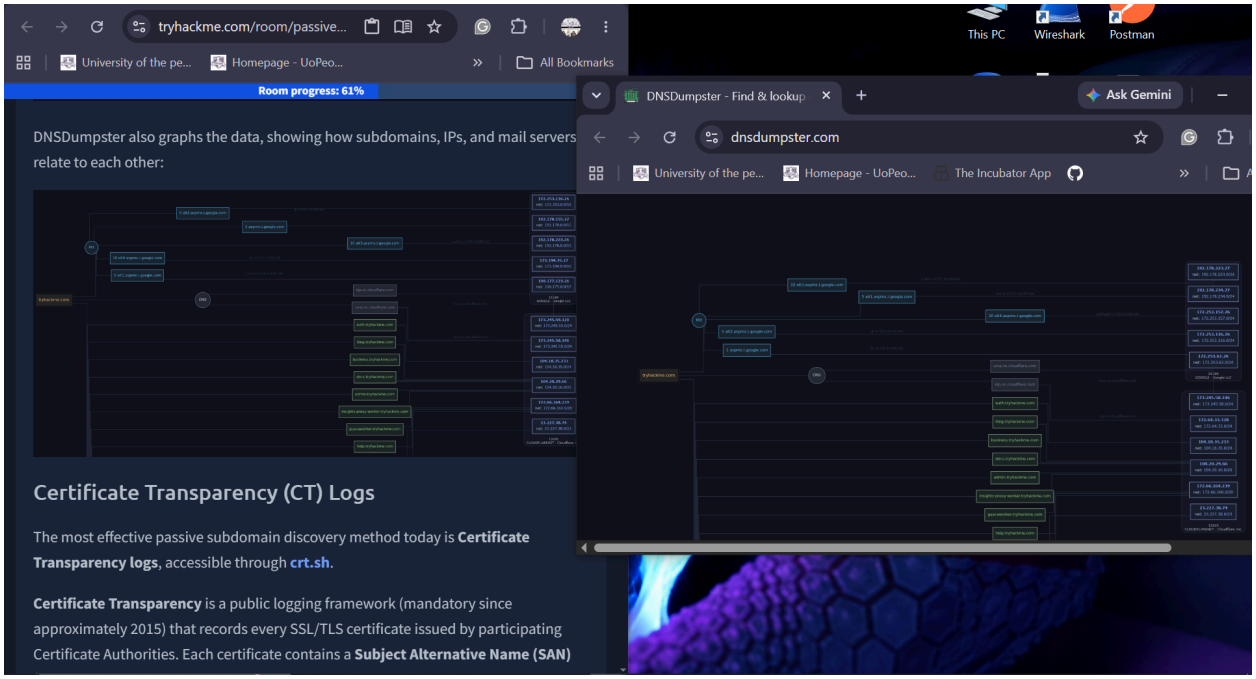
What I found for tryhackme.com on DNSDumpster:

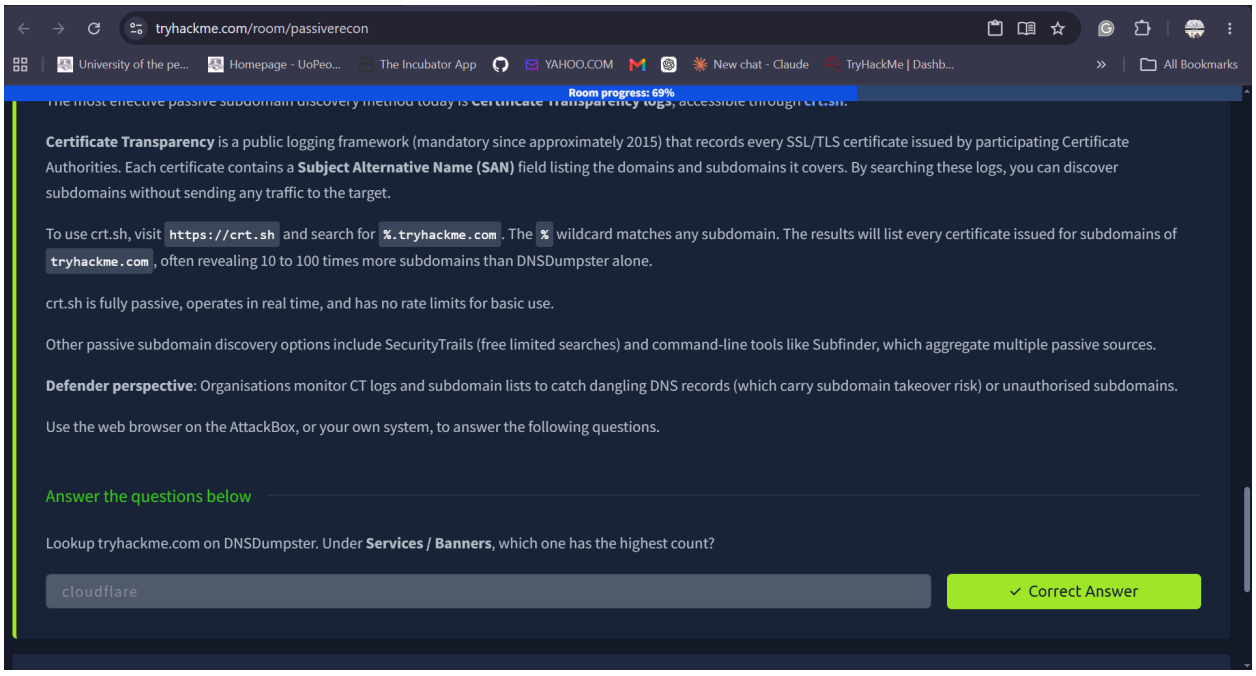
- Subdomains: admin, blog, docs, help, store, auth, careers, assets, status and others.
- Technology: most hosts sit behind **Cloudflare**, some use **Amazon CloudFront**, and others run **nginx 1.29.x** on AWS.
- Mail: Google (MX records). DNS: Cloudflare (NS records).
- The admin host returns **403 Forbidden**, meaning it exists but is protected.

Forgotten subdomains matter because they can expose outdated software, development panels or admin portals.

Screenshots:







Task 6: Shodan.io

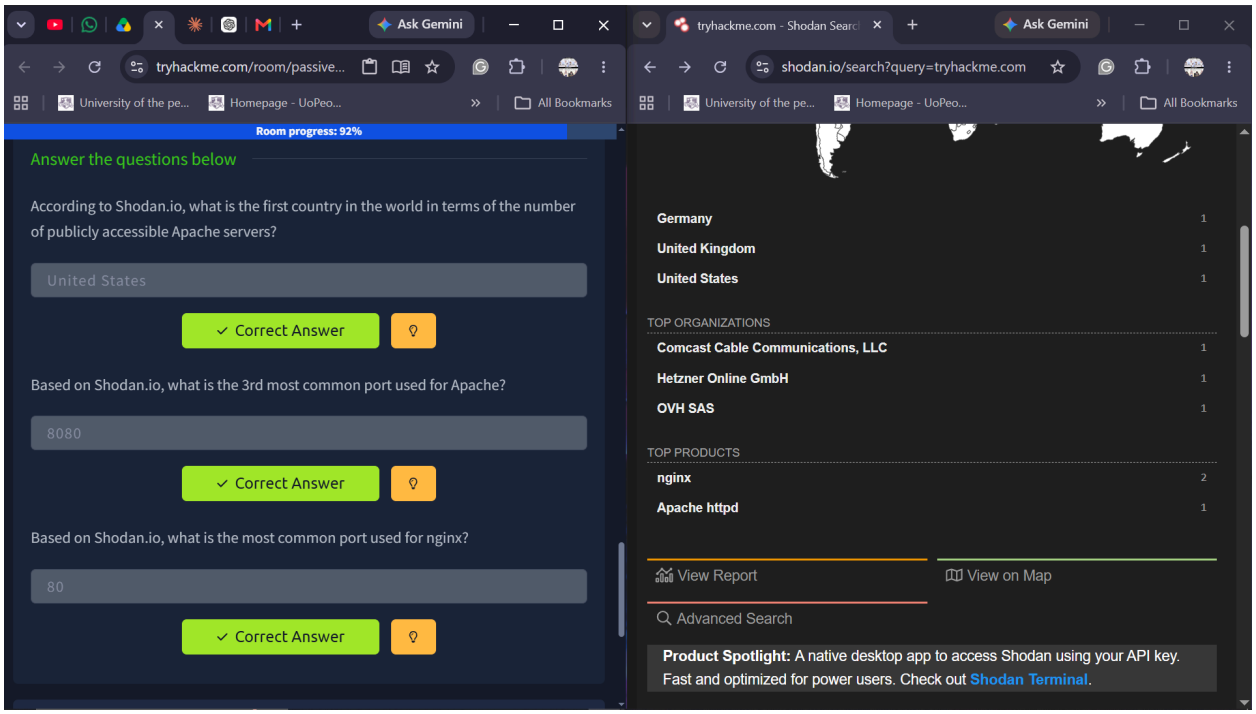
Notes:

Shodan is a search engine for internet-connected devices. It scans the internet and indexes banners, open ports, services, and software versions. A host page shows the IP and ASN, hosting provider, location, open ports with banners, and tags like cdn or vuln. Useful search filters:

- hostname:tryhackme.com
- org: "TryHackMe"
- port:443 country: US
- http.component: "wordpress"

Searching apache or nginx shows summary boxes such as **Top Countries** and **Top Ports**.

Screenshots:



Task 7: Summary

Notes:

This room covered fully passive techniques: WHOIS for registration details, DNS lookups for IPs, mail servers and TXT records, DNSDumpster and crt.sh for subdomains, and Shodan for exposed services. They trigger no alerts and carry

minimal legal risk when used within scope, yet they often reveal forgotten subdomains, outdated services, and misconfigurations. Results change over time because IPs rotate and subdomains appear and disappear.

Tools used

whois, nslookup, dig, DNSDumpster, crt.sh, Shodan.io, ping

Security observations to include

- **Version exposure:** DNSDumpster showed exact nginx versions (1.29.0 and 1.29.1) on several hosts, the same information disclosure issue as the Server header.
- **Large attack surface:** one domain exposed many subdomains (admin, status-api, reverse-proxy), all found without touching the target.
- **Defensive use:** organisations can monitor Shodan, certificate logs and DNS changes to find exposures before attackers do.

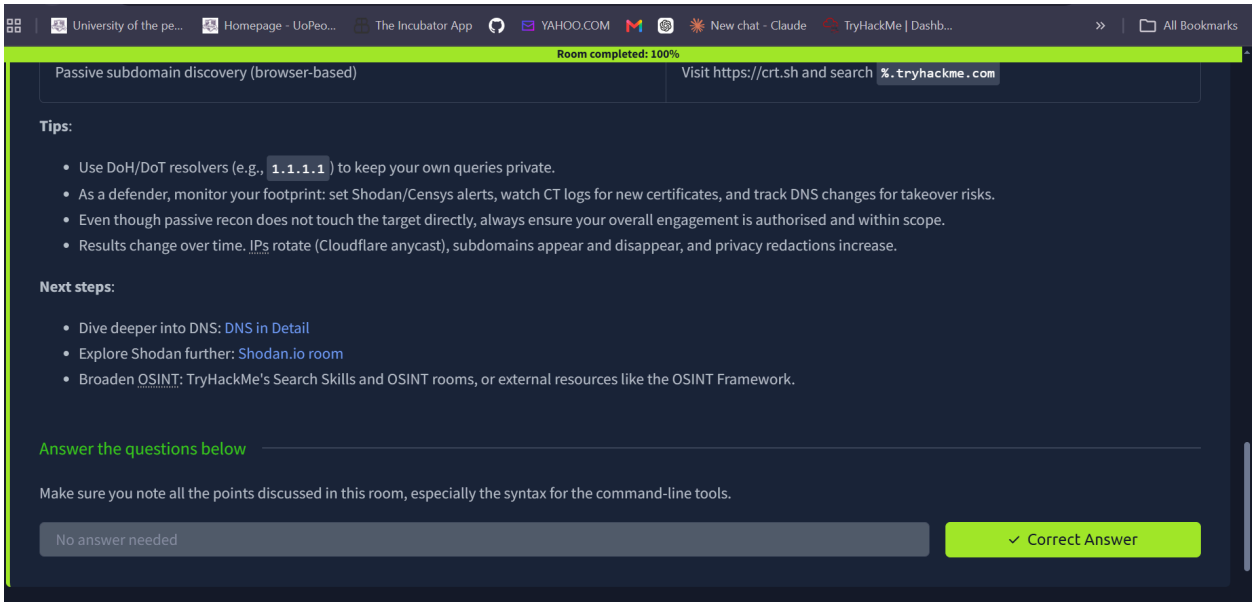
Final "What I learned" paragraph

This room taught me that a lot of valuable information about a target is publicly available without sending a single packet to it. I learned to read WHOIS records, query DNS record types with dig and nslookup, find subdomains through DNSDumpster and certificate transparency logs, and use Shodan to see what a target exposes. I also practised troubleshooting when the AttackBox had no internet access, which taught me to check connectivity before assuming a command is wrong.

Folder checklist for Room1_PassiveRecon

- **Notes:** everything above, in your own words
- **Screenshots:** one per task, every command with its output, the DNSDumpster and Shodan pages, and the room-completed screen
- **Completion link:** the room URL and your TryHackMe profile link

Screenshot:



Completion link:

https://tryhackme.com/room/passiverecon?utm_campaign=social_share&utm_medium=social&utm_content=share-completed-room&utm_source=copy&sharerId=6a298268d8b6b164348fab72

Room 2: Active Recon

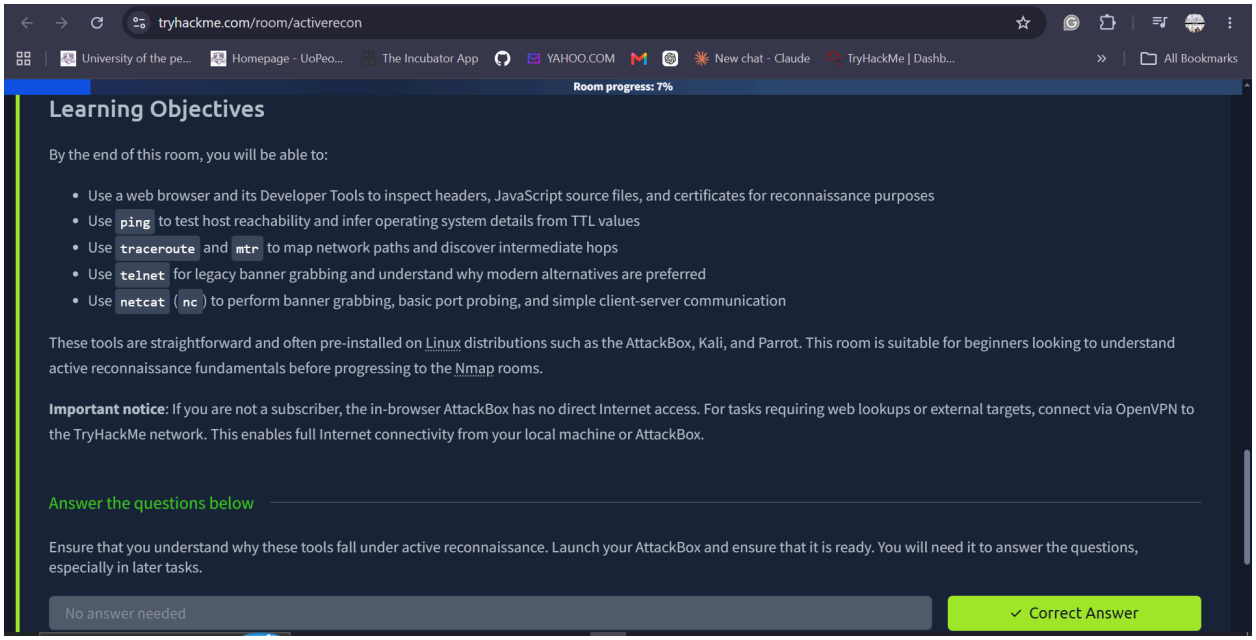
Platform: TryHackMe
Objective: Learn to gather information by interacting directly with a target, using browser Developer Tools, ping, traceroute, telnet and netcat to identify technologies, check reachability, map the network path and grab service banners.

Task 1: Introduction

Notes:

> Active recon means interacting directly with the target, so my activity can be logged, detected or blocked, and it needs authorisation. Passive recon uses only public data. This room covers five tools: the browser with Developer Tools, ping, traceroute, telnet and netcat.

Screenshot:



Task 2: Web Browser and Developer Tools

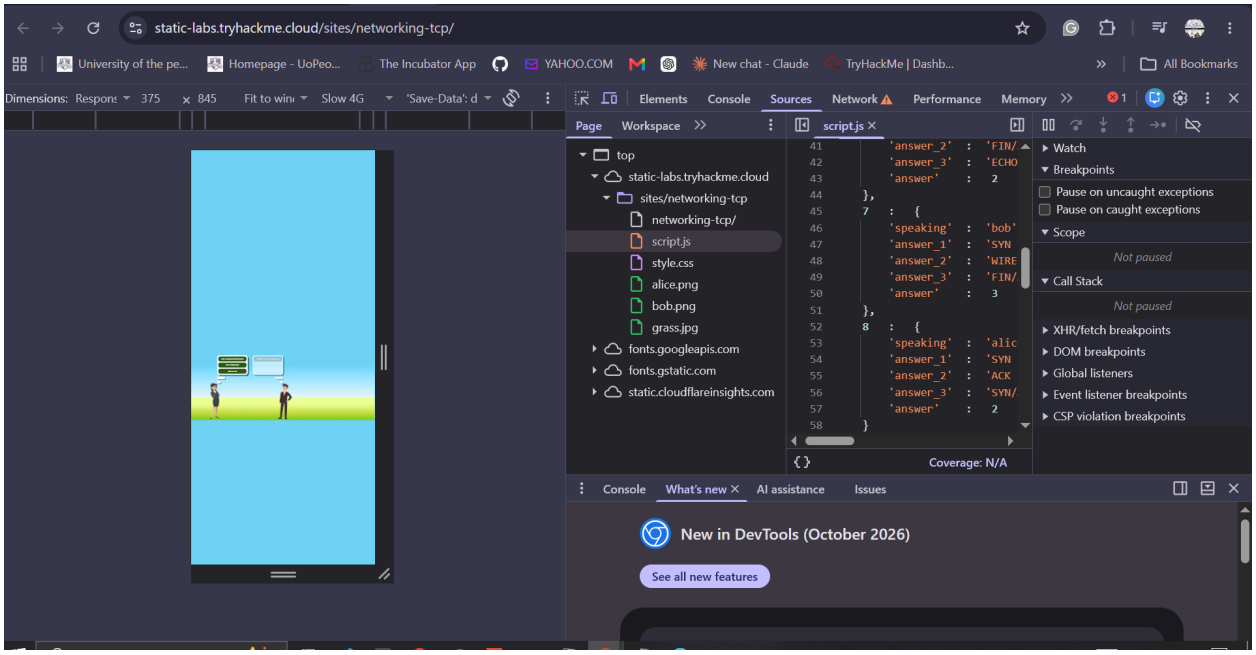
Notes:

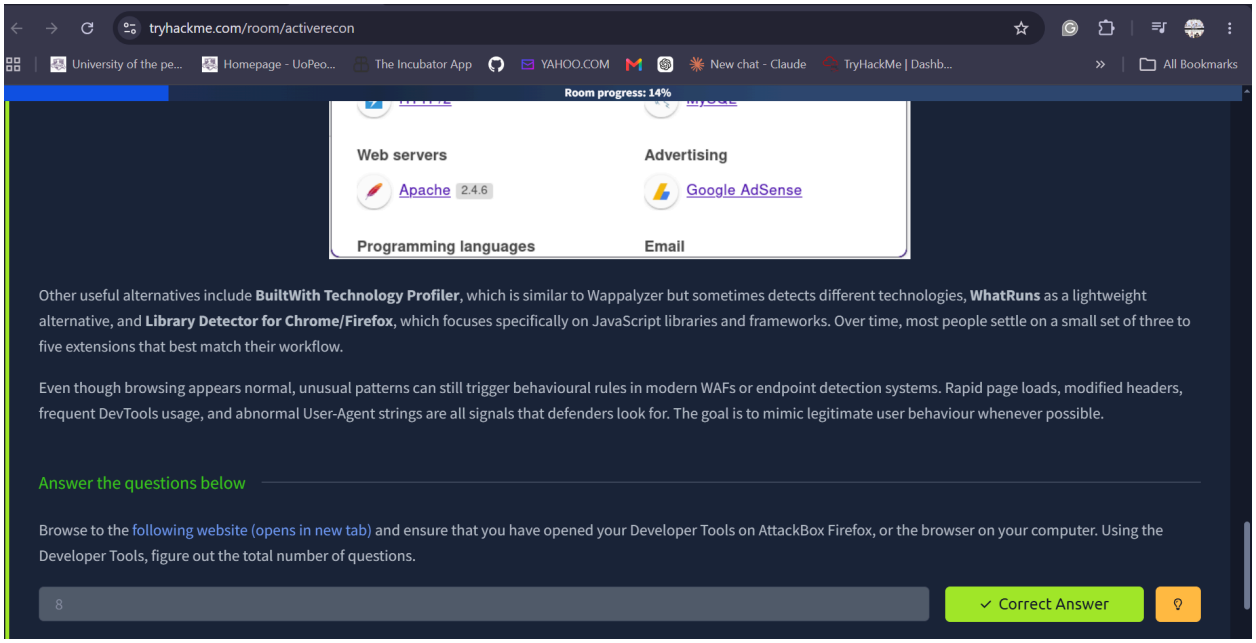
Developer Tools (F12, or Ctrl+Shift+I on Linux and Windows) let me inspect everything the server sends. The **Inspector/Elements** tab shows the page structure, the **Network** tab shows requests, responses, and headers, and the **Debugger/Sources** tab shows the JavaScript files. This can reveal server technologies, headers, hidden endpoints, comments, and certificate details.

What I did:

I opened the static site, used Developer Tools to read the page's JavaScript, and counted the total number of questions: **[your number]**.

Screenshots:

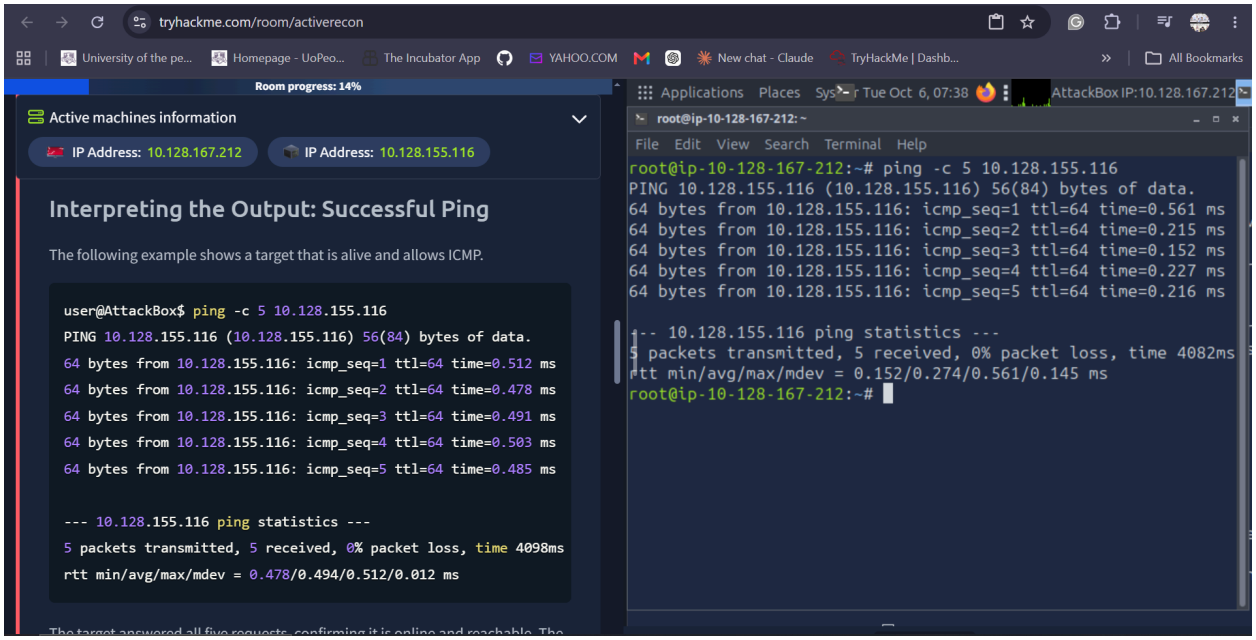




Task 3: Ping

Notes:

ping sends ICMP echo requests and waits for echo replies, which shows whether a host is reachable. The reply's **TTL** value can hint at the operating system. Useful options:



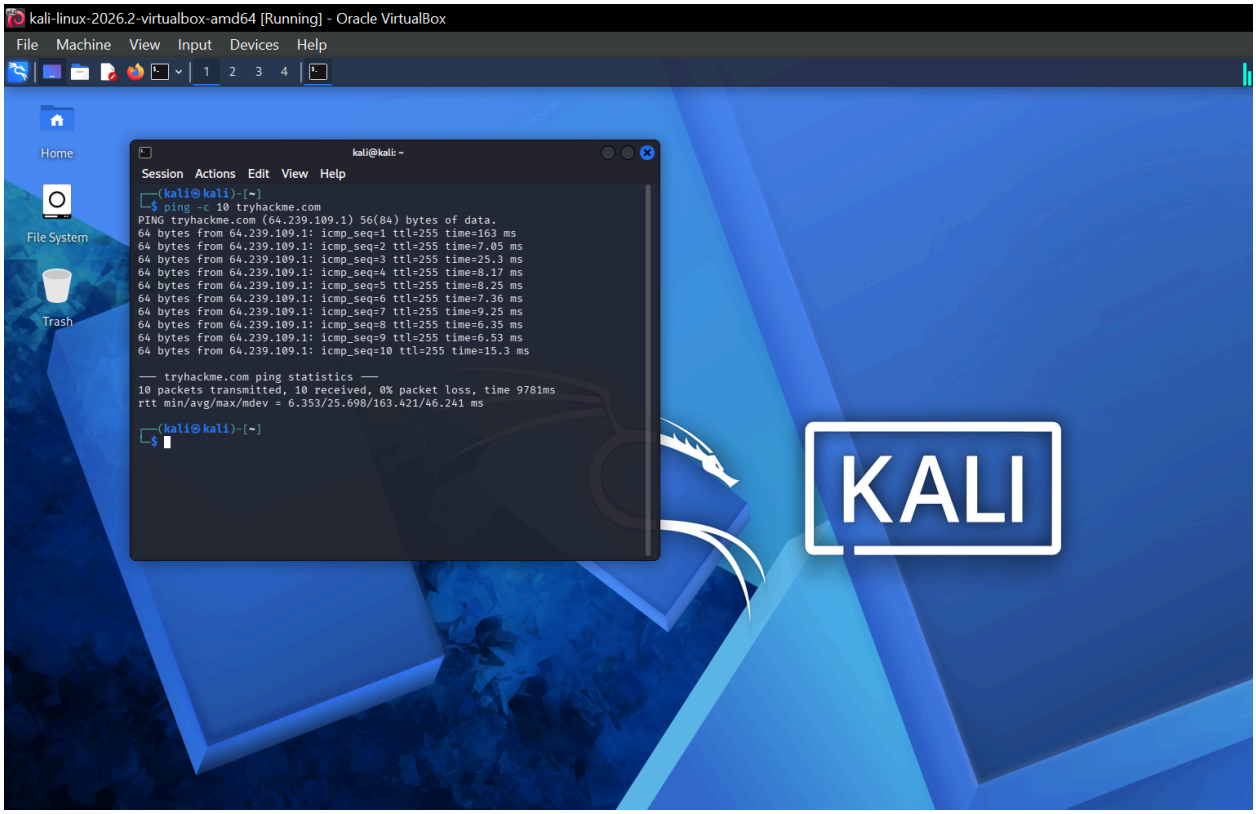
Options	Purpose
-c N	Send N packets, then stop
-s SIZE	Set the size of the data in each packet
-i SECONDS	Wait between packets
-W SECONDS	Wait time for a reply

The ICMP header is **8 bytes**. Windows Firewall **blocks ping by default**, so a missing reply doesn't prove a host is down.

Command used:

ping -c 10 <MACHINE_IP>

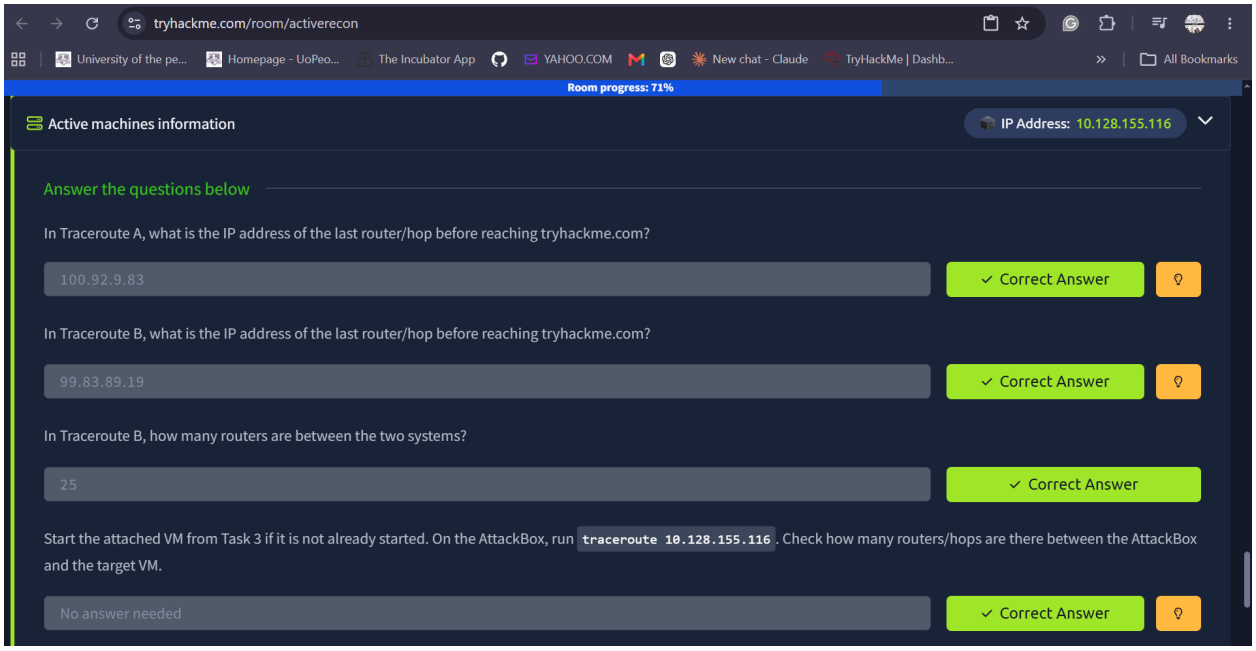
Result: [number] replies out of 10.



Problem-solving note:

> My first ping from my Kali VM showed 100% packet loss. The target had a private 10.128.x.x address on TryHackMe's network, and my VM wasn't connected to it.
> **[Fix that worked: used the AttackBox / connected OpenVPN.]**

Screenshots:



Task 4: Traceroute

Notes:

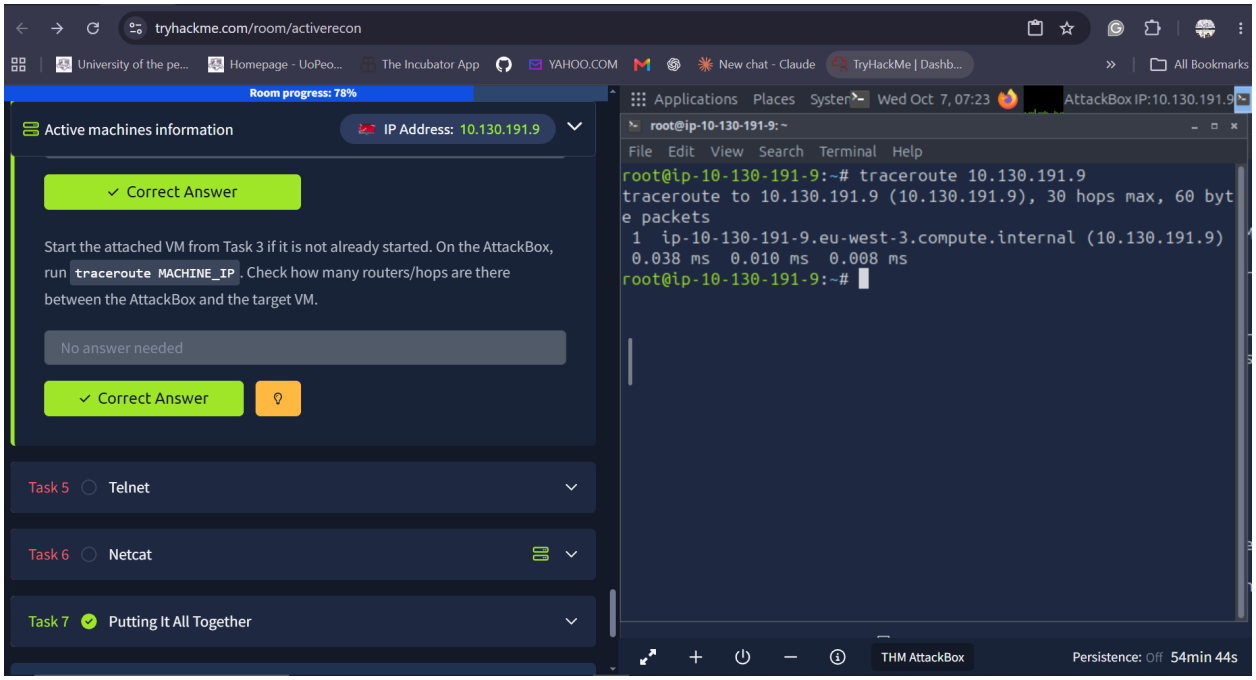
traceroute maps the path packets take to a target. It sends packets with increasing TTL values. Each router reduces the TTL by one, and when it reaches zero the router sends back an ICMP "time exceeded" message that reveals its IP. A * means no reply for that probe, often because of a firewall. Several IPs on one hop mean traffic takes different paths (load balancing).

Counting rule: the last hop is the destination, so the routers between me and the target are the last hop number minus 1. In Traceroute B the target was hop 26, so there were **25 routers** and the last router was 99.83.89.19.

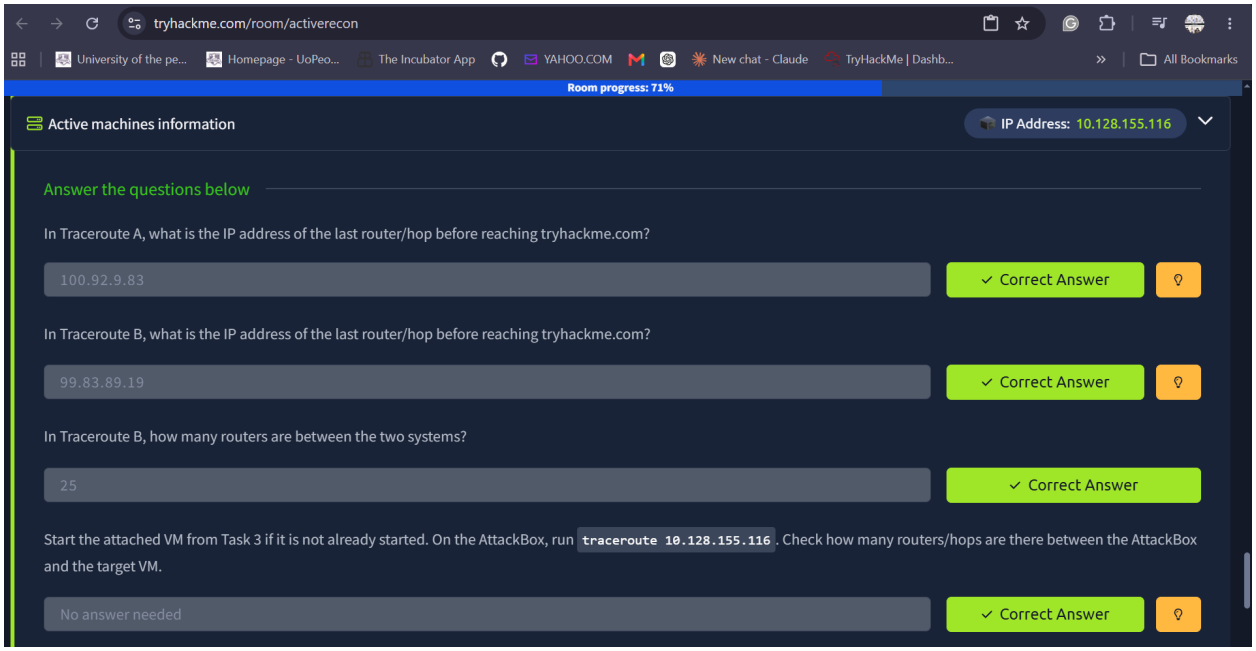
Commands used:

traceroute tryhackme.com

traceroute <MACHINE_IP>



Screenshots:



Task 5: Telnet

Notes:

telnet (TCP port 23 by default) was built for remote administration, but it sends everything in **cleartext**, including passwords, so **SSH** replaced it. Because it works

over TCP, I can connect to any port and read the service's **banner**, which often reveals the software name and version. This is **banner grabbing**. Versions can be checked against CVE and Exploit-DB. Telnet can't handle encrypted services such as HTTPS, so curl -I https://MACHINE_IP or openssl s_client -connect MACHINE_IP:443 are used instead.

Commands used:

telnet <MACHINE_IP> 80

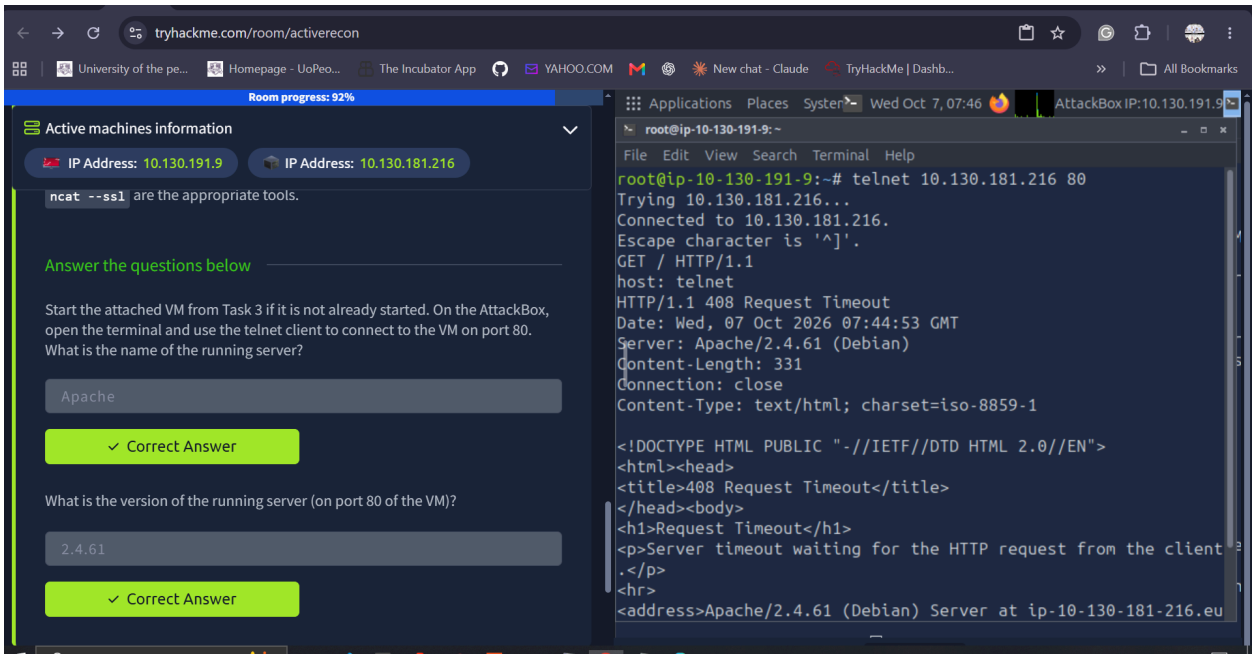
GET / HTTP/1.1

host: telnet

(press Enter twice)

Result: the Server: header showed **[server name]** version **[version]**.

Screenshots:



Task 6: Netcat

Notes:

netcat (nc) also connects to any TCP port, and it is preferred over telnet because it is more flexible. It can act as a **client** (nc MACHINE_IP PORT) to grab banners, or as

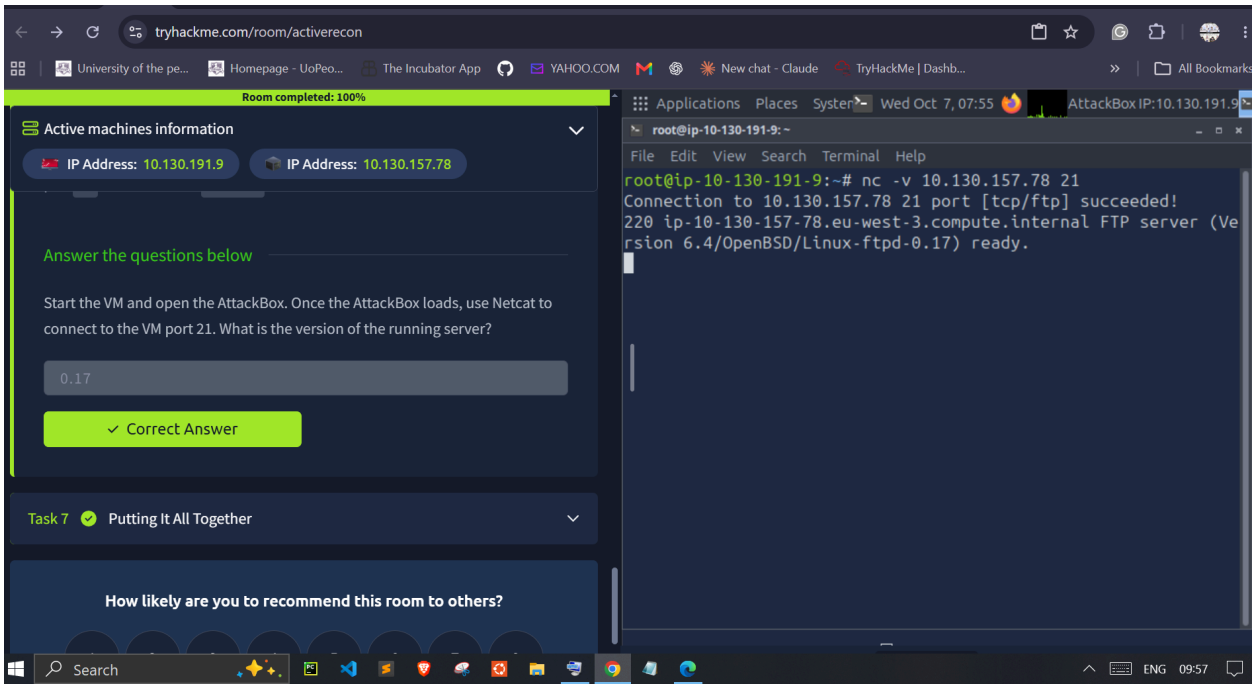
a **server** (nc -lvnp PORT) that listens for connections. **[Add what your room's tasks and questions covered.]**

Commands used:

nc <MACHINE_IP> <PORT>

nc -lvnp <PORT>

Screenshots:



Task 7: Putting It All Together

Notes:

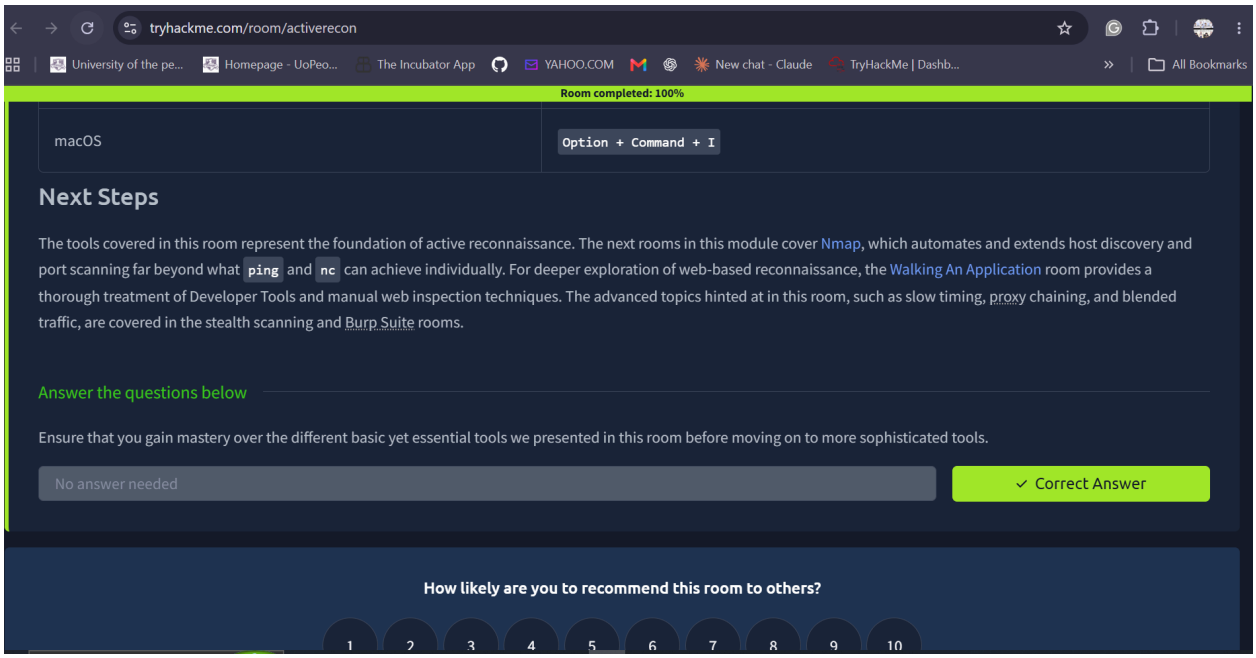
The tools work best in sequence. ping confirms a host is alive, traceroute shows the network path and possible filtering, and nc or telnet probe specific ports for services and versions. For HTTP, curl -I and nc are preferred over telnet. The next step is Nmap, which automates host discovery and port scanning.

Quick reference:

Purpose	**Command**

Ping (Linux)	ping -c 10 MACHINE_IP
Ping (Windows)	ping -n 10 MACHINE_IP
Traceroute (Linux)	traceroute MACHINE_IP
Traceroute (Windows)	tracert MACHINE_IP
Live path monitoring	mtr MACHINE_IP
Banner grab (telnet)	telnet MACHINE_IP PORT
Banner grab (netcat)	nc MACHINE_IP PORT
HTTP headers	curl -I http://MACHINE_IP

Screenshot:



Tools used

Browser Developer Tools, ping, traceroute, telnet, netcat/curl, OpenVPN **[if you used it]**, TryHackMe AttackBox / Kali VM

Security observations to include

- **Version exposure:** the Server header on the target revealed its software and version, which an attacker can match to known vulnerabilities.
- **Cleartext protocols:** telnet sends credentials unencrypted, so it should be replaced by SSH.
- **Filtering clues:** * hops and blocked pings show that firewalls limit what recon can see, and that a silent host isn't always offline.
- **Detectability:** every tool here touches the target directly, so it needs authorisation.

Final "What I learned" paragraph

> This room showed me how active recon builds a picture of a target by interacting with it directly. I learned to inspect a site with Developer Tools, test reachability with ping, map the path with traceroute, and grab service banners with telnet and netcat. I also learned to read failures: 100% packet loss and rows of * didn't mean the target was broken; they meant my VM wasn't connected to the right network. That taught me to check connectivity before blaming a command.

Completion link:

https://tryhackme.com/room/activerecon?utm_campaign=social_share&utm_medium=social&utm_content=share-completed-room&utm_source=copy&sharerId=6a298268d8b6b164348fab72

Room 3: Content Discovery

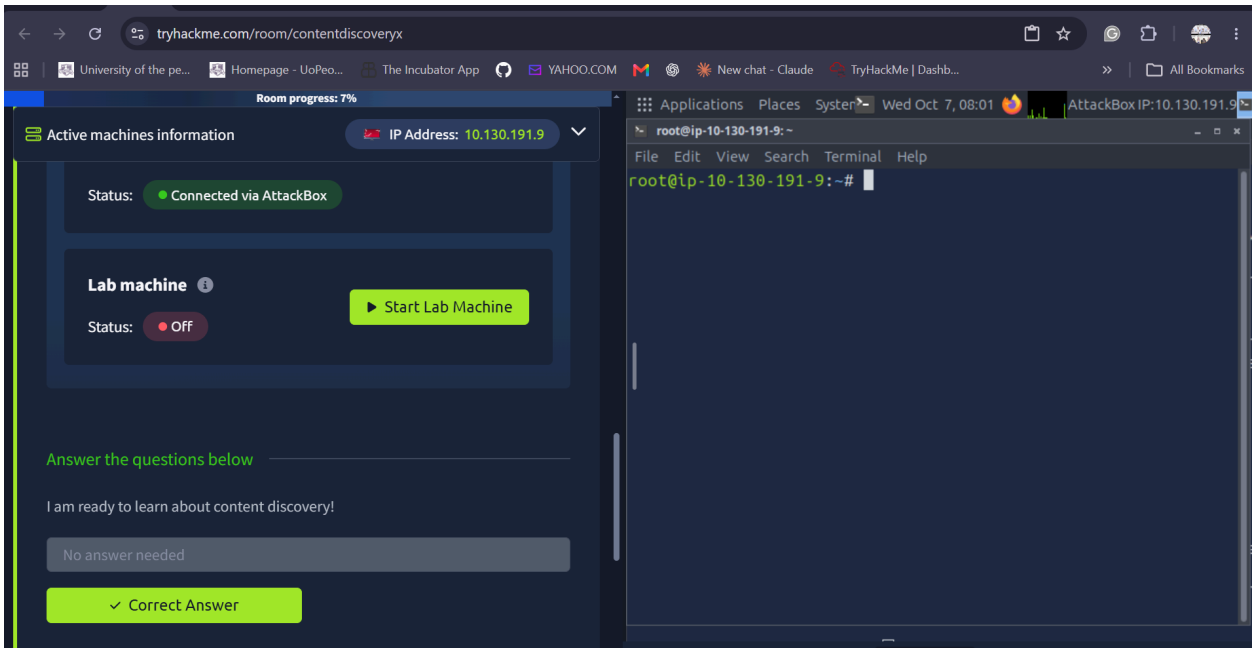
Platform: TryHackMe
Objective: Learn to find content on a web server that isn't meant to be public or isn't linked anywhere obvious (pages, files, directories, admin portals, config files, and backups) using manual, OSINT, and automated techniques.

Task 1: Introduction

Notes:

Content is anything hosted on a web server, such as pages, files, directories, admin portals, configuration files and backup archives. **Content discovery** is finding the content that wasn't meant to be public or isn't linked obviously, like staff portals, old site versions, exposed backups and admin panels. There are three approaches: **manual**, **automated** and **OSINT**. This room covers all three, ending with Gobuster.

Screenshot:



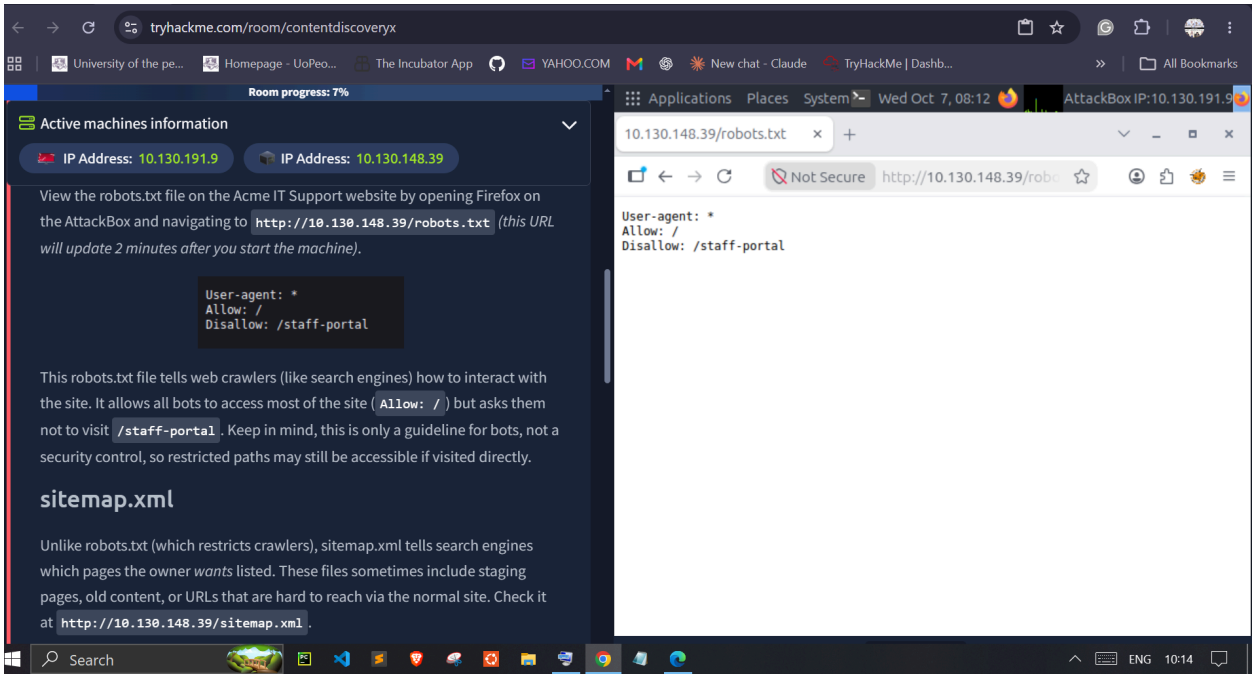
Task 2: Manual Discovery - Common Files

Notes:

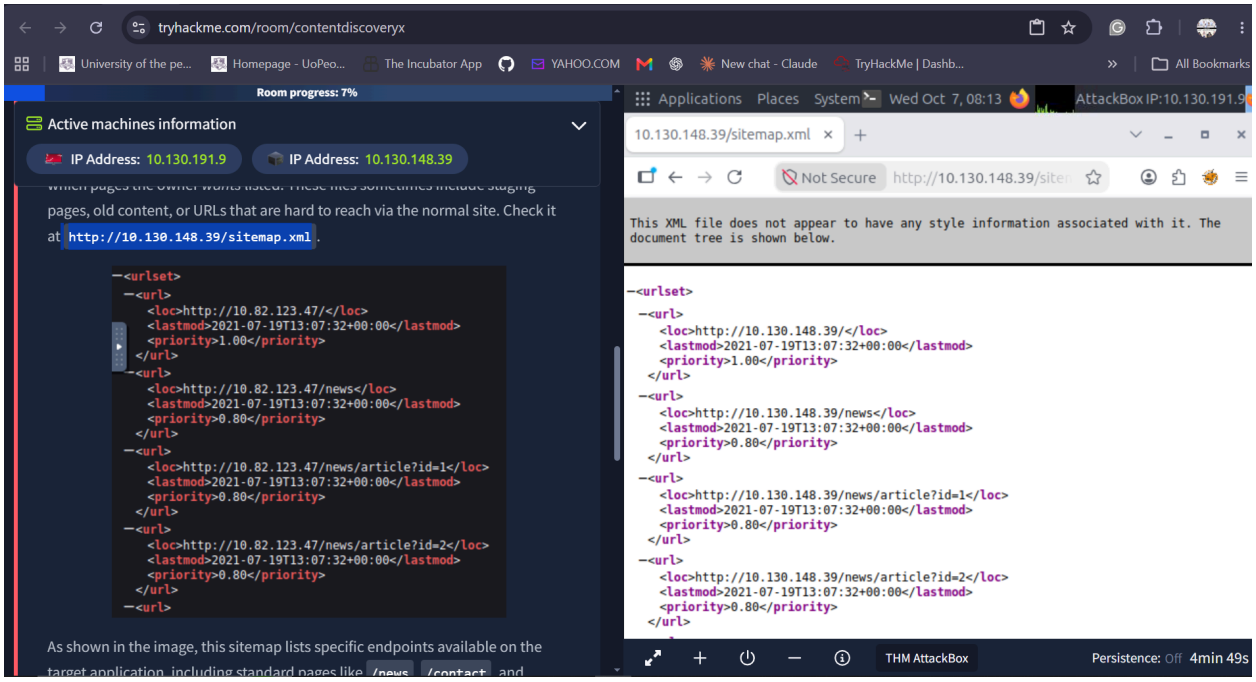
- **robots.txt** tells search engine crawlers which pages they may or may not visit. Owners often list sensitive paths here, so it works as a ready-made list of interesting locations. On the Acme IT Support site, it asked bots not to visit /staff-portal. It's only a guideline for bots, not a security control, so the path may still be reachable directly.
- **sitemap.xml** lists the pages the owner wants indexed. It can include staging pages, old content, and hard-to-reach URLs. On the target, it listed /news, /contact, articles like /news/article?id=1, and more interesting paths such as /customers/login and /s3cr3t-area. The id= parameters also show possible input points to test later.

Commands/URLs used:

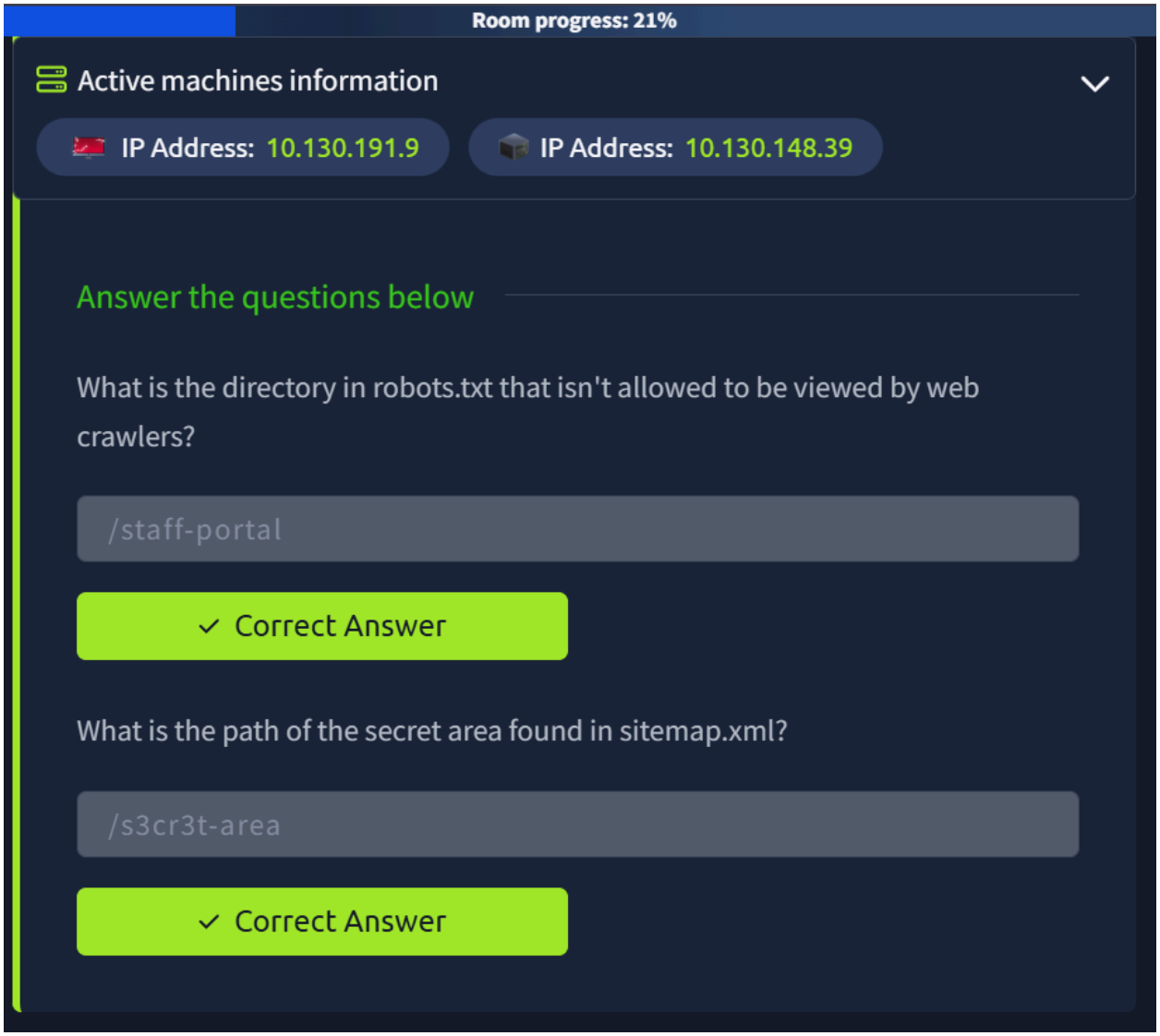
http://<TARGET_IP>/robots.txt



http://<TARGET_IP>/sitemap.xml



Screenshots:



Task 3: Manual Discovery - Headers and Framework Stack

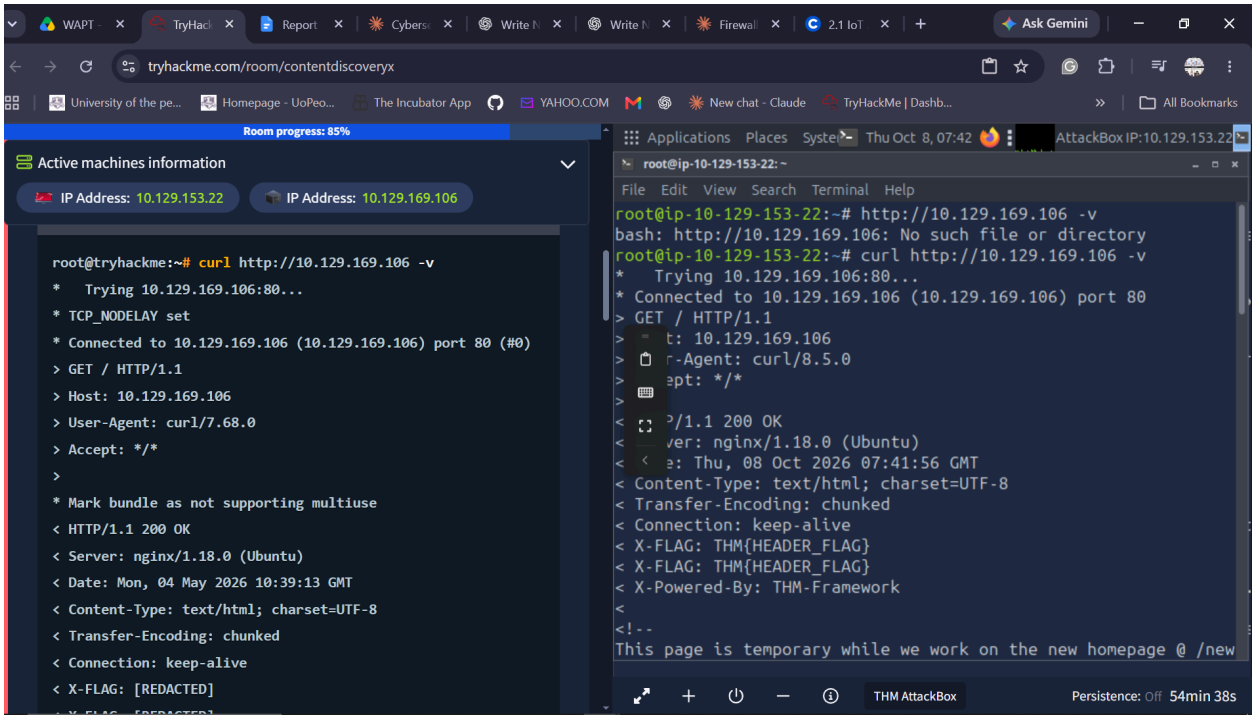
Notes:

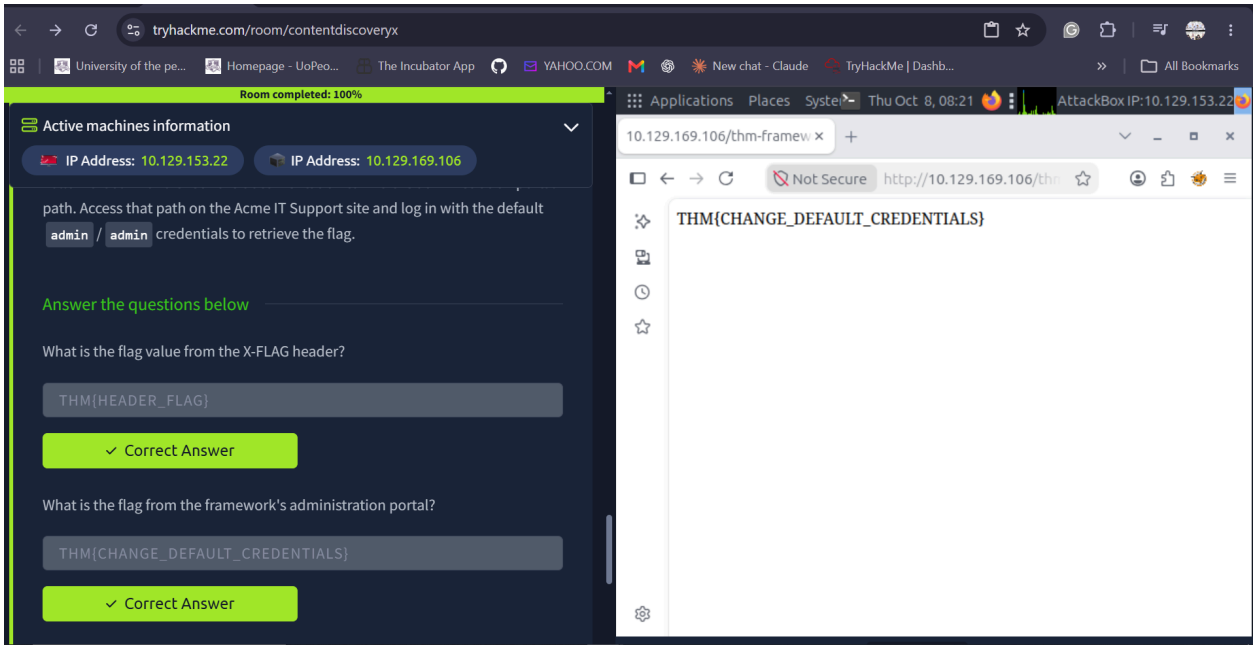
HTTP headers such as Server and X-Powered-By can reveal the web server software and the framework. Running curl -v against the site showed Server: nginx/1.18.0 (Ubuntu), X-Powered-By: THM-Framework, and custom X-FLAG headers. The page source also had comments: one mentioned a hidden page (/new-home-beta) and another at the bottom linked to the framework's website.

Following that link, the framework's documentation gave the admin portal path /thm-framework-login and the default login admin / admin. Logging in there gave the second flag. Identifying the framework led me to hidden paths and default credentials.

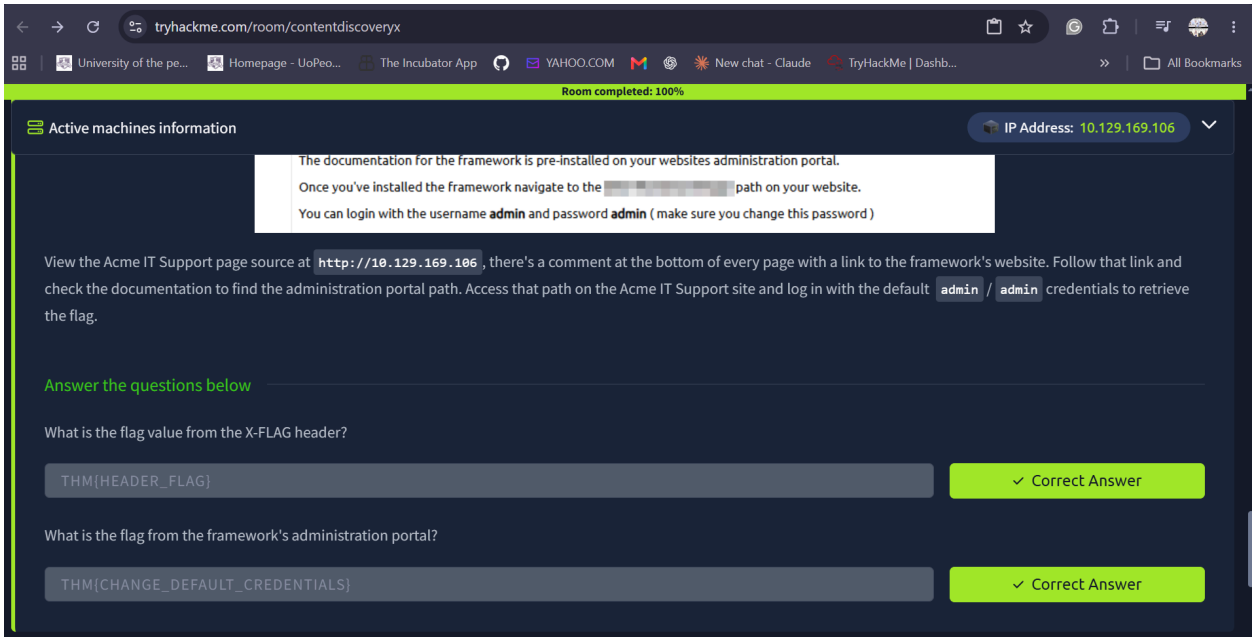
Commands used:

curl http://<TARGET_IP> -v





Screenshots:



Task 4: OSINT - Search Engines and Web Tools

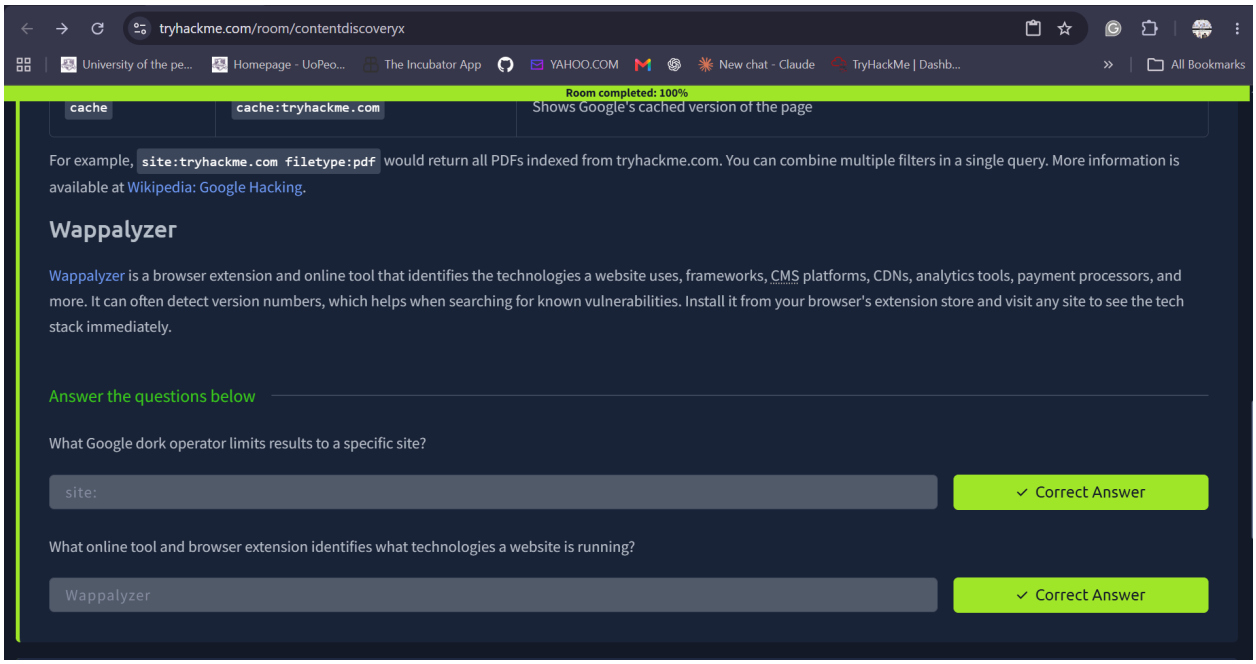
Notes:

Google dorking uses advanced search operators to find things the owner never meant to be public, such as admin panels, leaked documents and login pages:

Operator	Example	Purpose
site:	site:tryhackme.com	Results only from one domain
inurl:	inurl:admin	A word in the URL
filetype:	filetype: pdf	A specific file type
intitle:	intitle: admin	A word in the page title
intext:	intext: password	A word in the page body
cache:	cache:tryhackme.com	Google's cached copy

Operators can be combined, for example, site:tryhackme.com filetype: pdf.
Wappalyzer is a browser extension and online tool that identifies a site's technologies (frameworks, CMS, CDNs, analytics) and can often detect version numbers.

Screenshots:

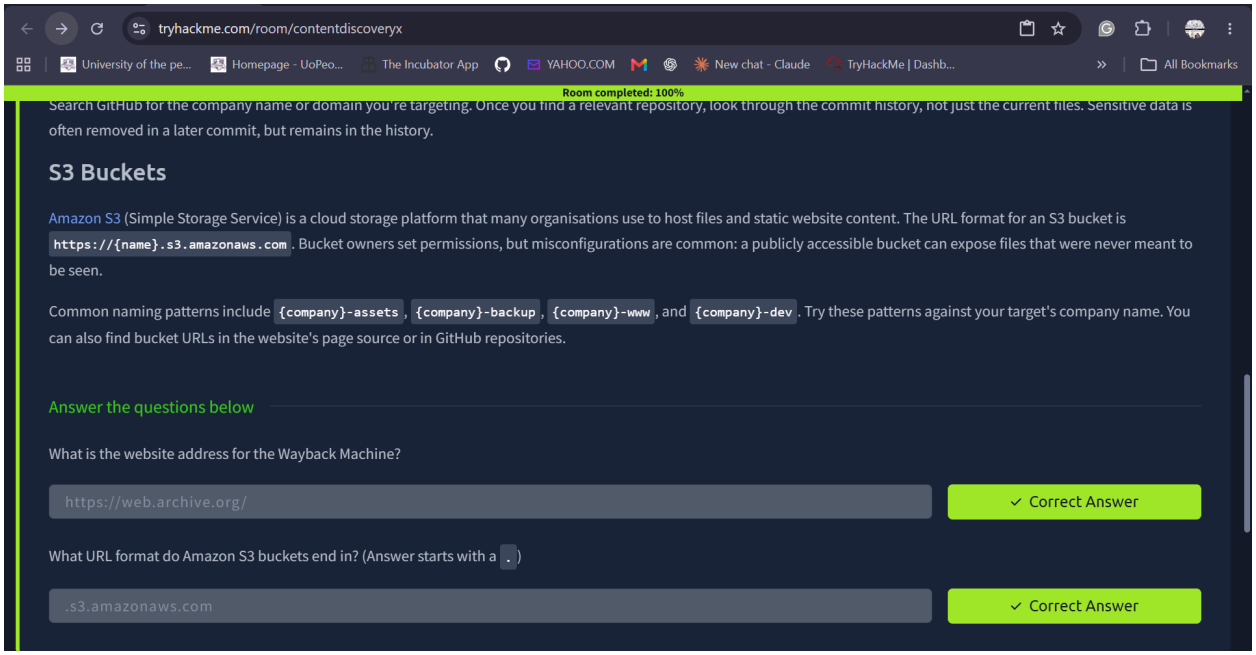


Task 5: OSINT - Repositories and Archives

Notes:

- **Wayback Machine** (web.archive.org) archives snapshots of the internet back to the late 1990s. It can show pages removed from the live site, such as old login forms and forgotten API endpoints.
- **GitHub** repositories sometimes contain accidentally committed API keys, credentials, config files and .env files. I should check the **commit history**, because data deleted in a later commit may still be there.
- **S3 buckets** are Amazon cloud storage with URLs in the form https://{name}.s3.amazonaws.com. Misconfigured public buckets can expose files. Common names follow patterns like {company}-assets, {company}-backup, {company}-www and {company}-dev. Bucket URLs may also appear in page source or in GitHub.

Screenshots:



Task 6: Automated Discovery - Gobuster Fundamentals

Notes:

Automated discovery sends hundreds or thousands of requests using a **wordlist** to see which directories and files exist. **Gobuster** has three modes: dir, dns, and vhost. I used dir mode with the SecLists common.txt wordlist. It hides 404 results, so everything shown is a path that exists.

Flag	**Purpose**
-u	Target URL
-w	Wordlist(required)
-t	Number of threads
-x	File extensions to try
-s	Only show certain status codes
-o	Save output to a file

--delay	Wait between requests

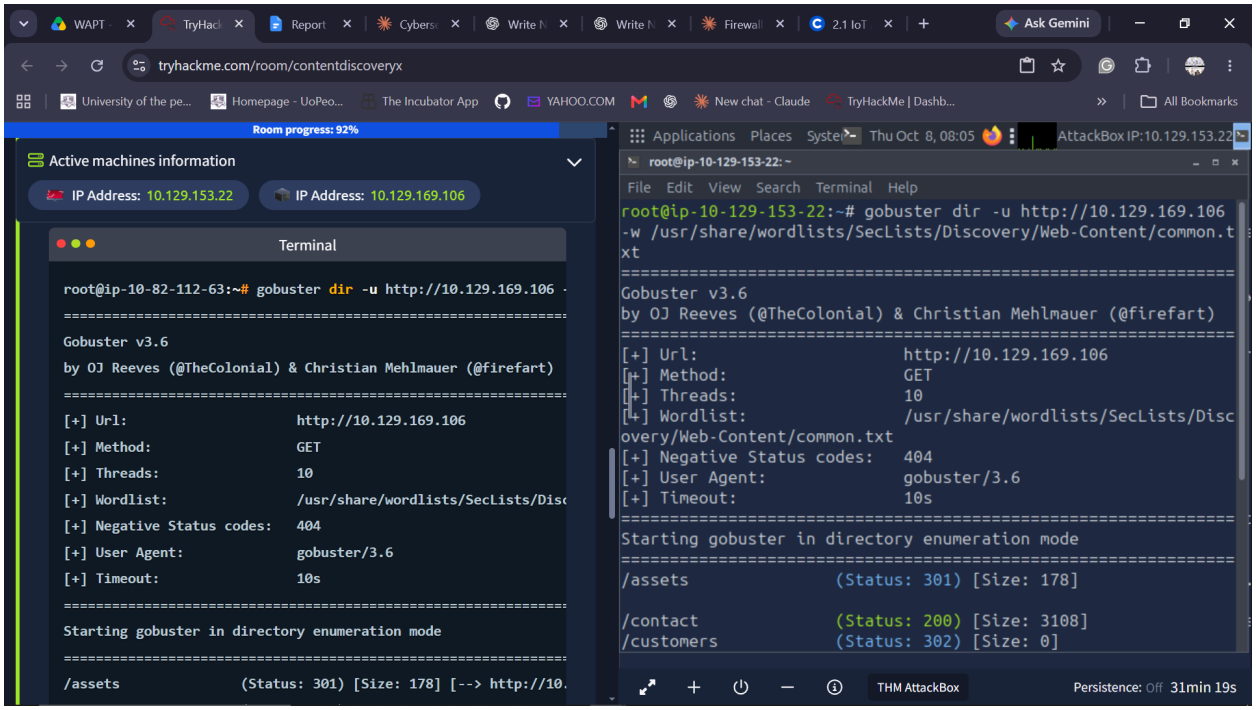
What was found:

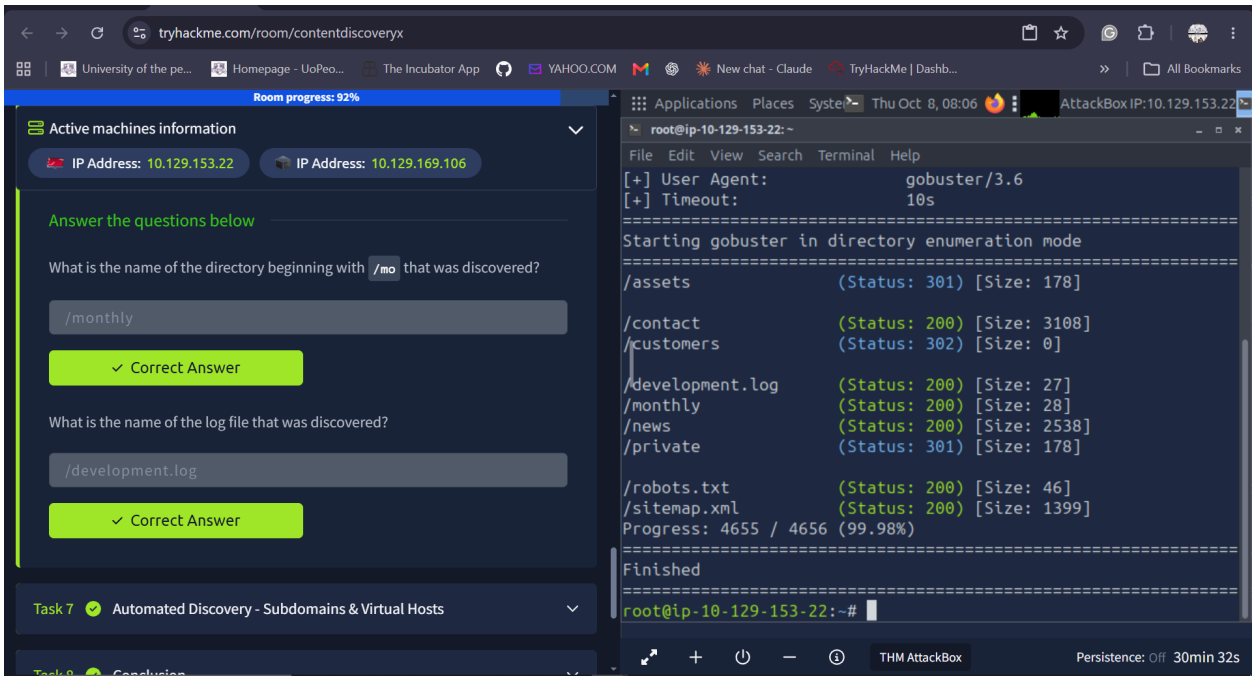
Path	**Status**	**Why it's interesting**
/assets	301	Static files
/contact, /news	200	Normal pages
/customers	302	Redirects to a login page
/development.log	200	A leftover log file that may hold sensitive data
/monthly	200	A page not linked from the menu
/private	301	A directory the owner may not want public
/robots.txt, /sitemap.xml	200	Files listing more paths

Command used:

gobuster dir -u "http://<TARGET_IP>" -w /usr/share/wordlists/SecLists/Discovery/Web-Content/common.txt

Screenshots:





Task 7: Automated Discovery - Subdomains and Virtual Hosts

Notes:

A **subdomain** is resolved through **DNS**. A **virtual host** is resolved by the **web server**, which uses the Host: header to choose which of several sites on one IP to

serve. Subdomains matter because something patched on the main domain may still be vulnerable on a subdomain.

Gobuster	**Purpose**
dns	Brute-forces subdomains. Required flags: -d (domain) and -w (wordlist)
vhost	Cycles wordlist entries through the Host: header to find hosts not in public DNS

Useful flags: -i (show IPs), -r (custom resolver), --wildcard

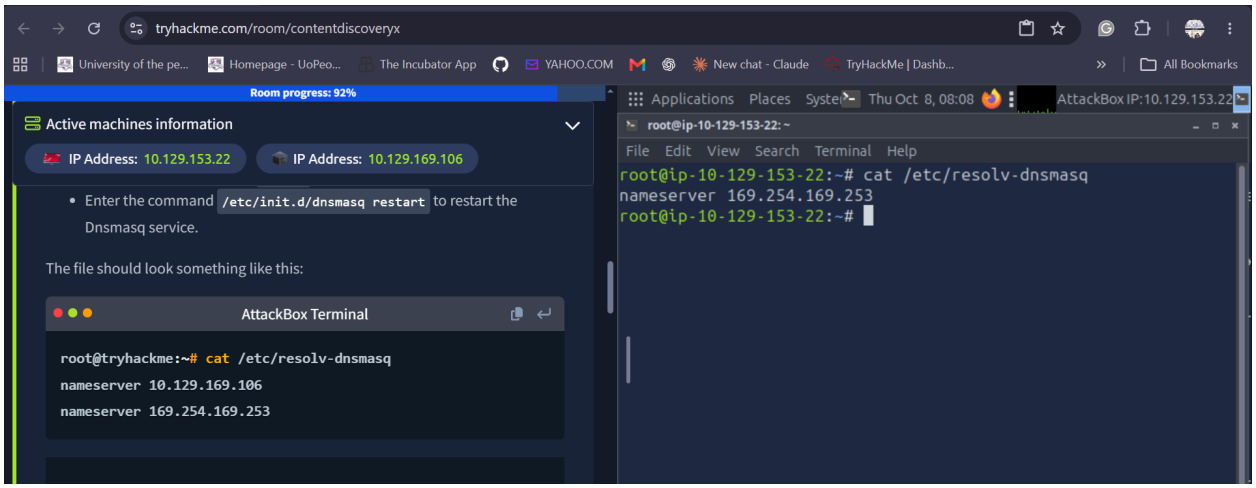
(continue when wildcard DNS is detected), --append-domain

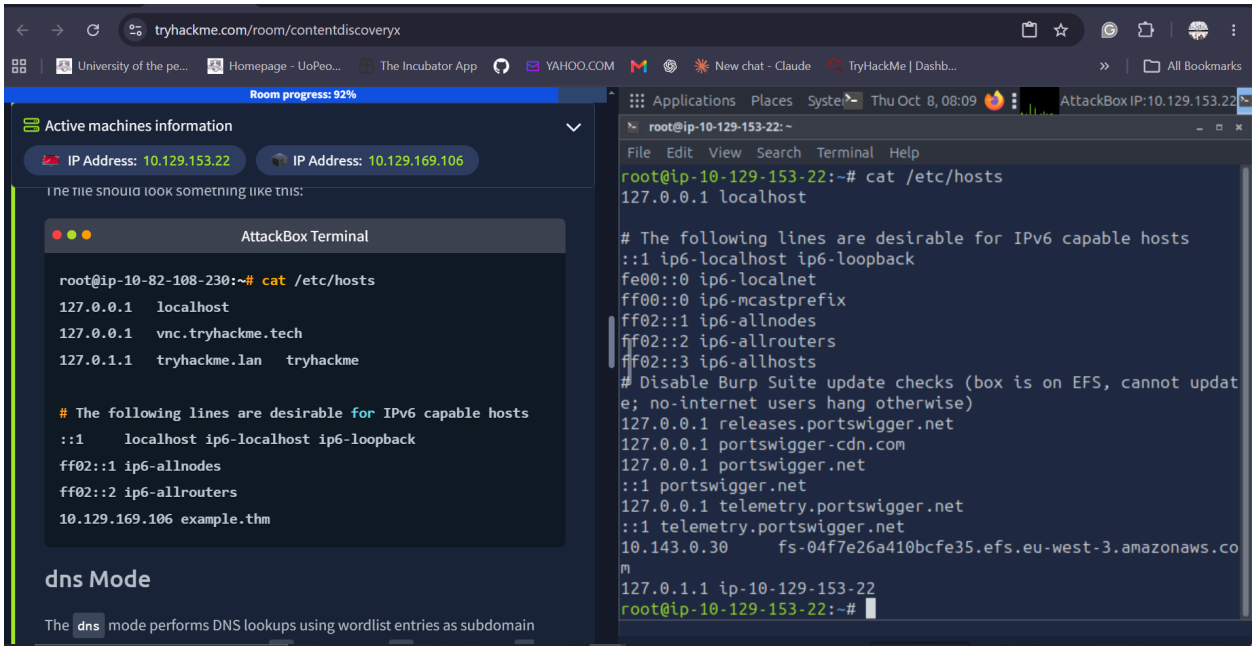
(combine each entry with the domain) and --exclude-length

(hide false positives of a given size). To make the lab domain resolve, I mapped it to the target IP in /etc/hosts.

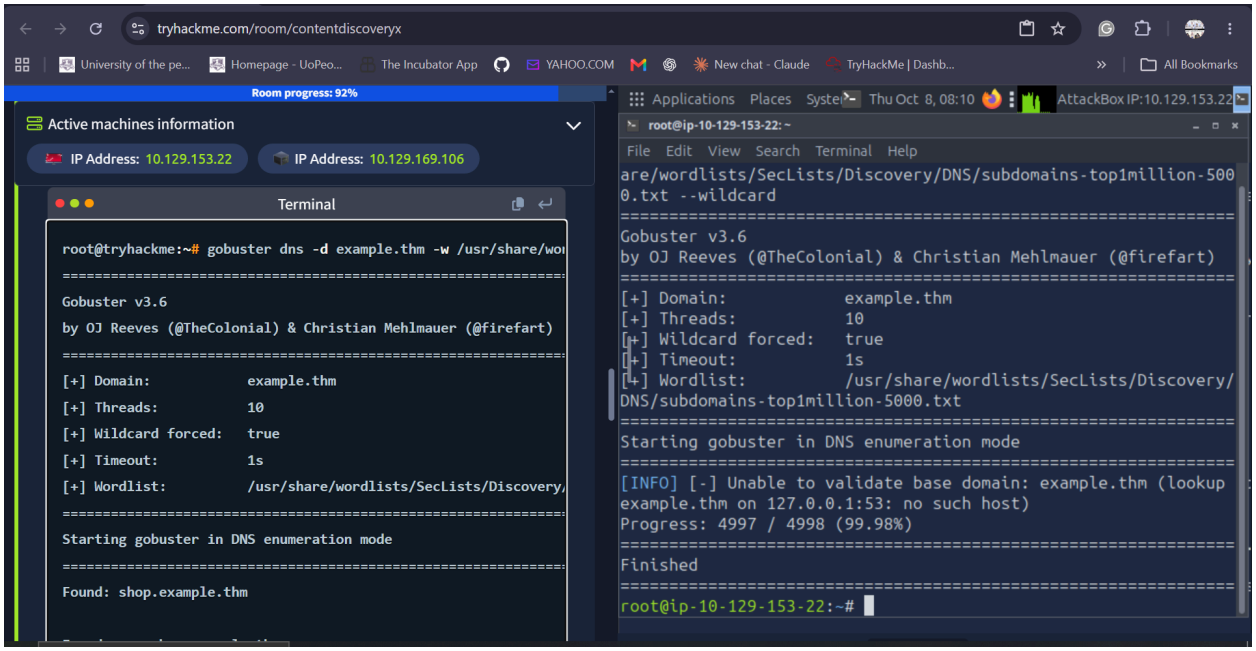
Commands used:

sudo nano /etc/hosts (added: <TARGET_IP> acmeitsupport.thm)

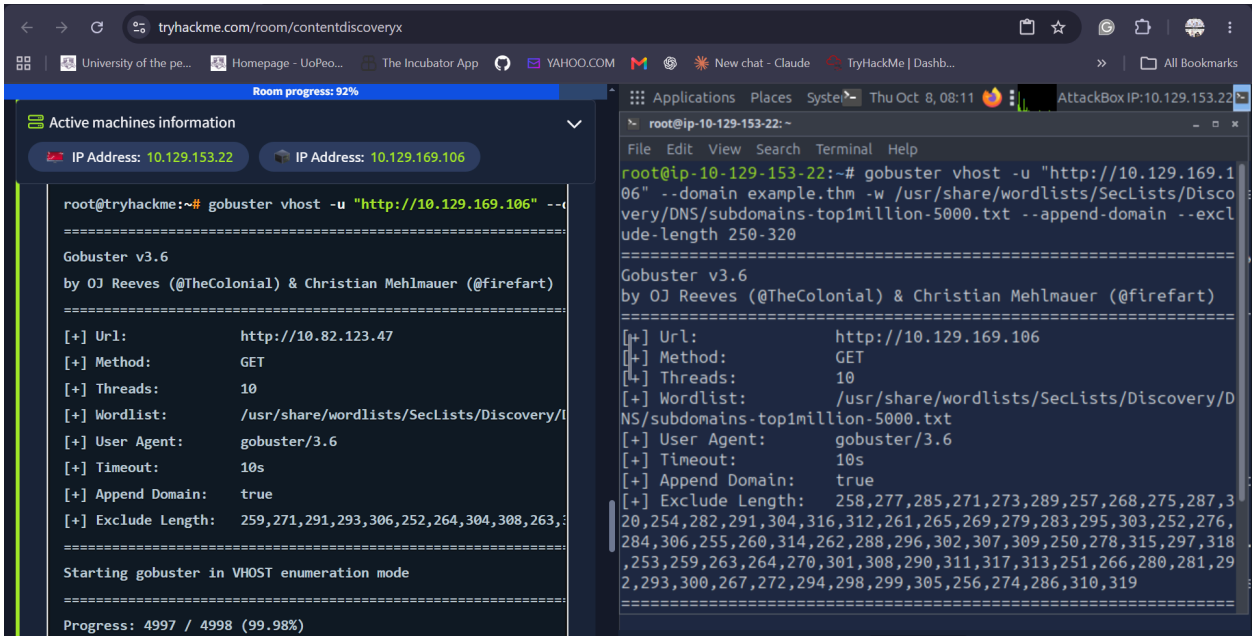




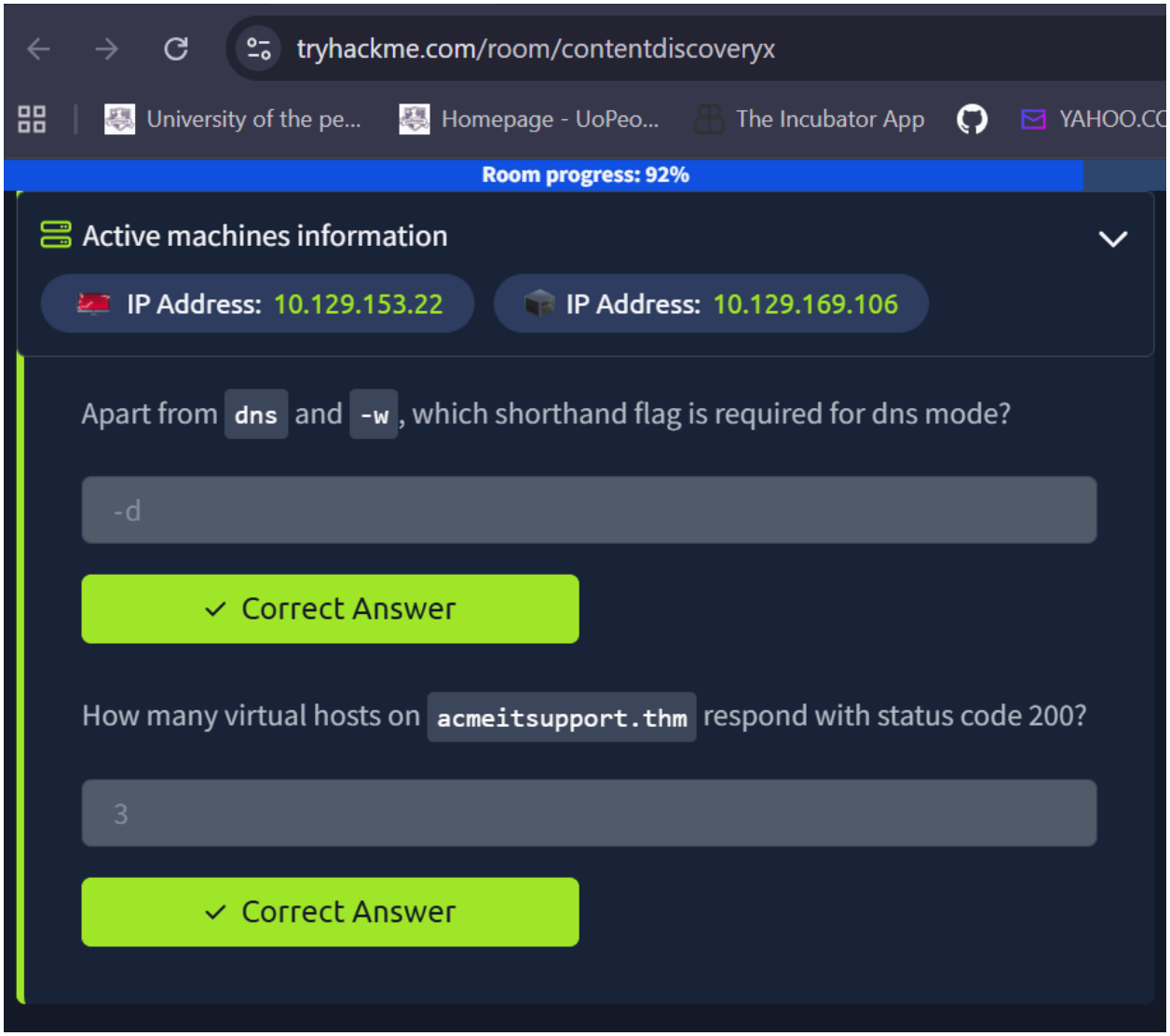
gobuster dns -d example.thm -w <wordlist> --wildcard



gobuster vhost -u "http://<TARGET_IP>" --domain acmeitsupport.thm -w <wordlist> --append-domain



Screenshots:



Task 8: Conclusion

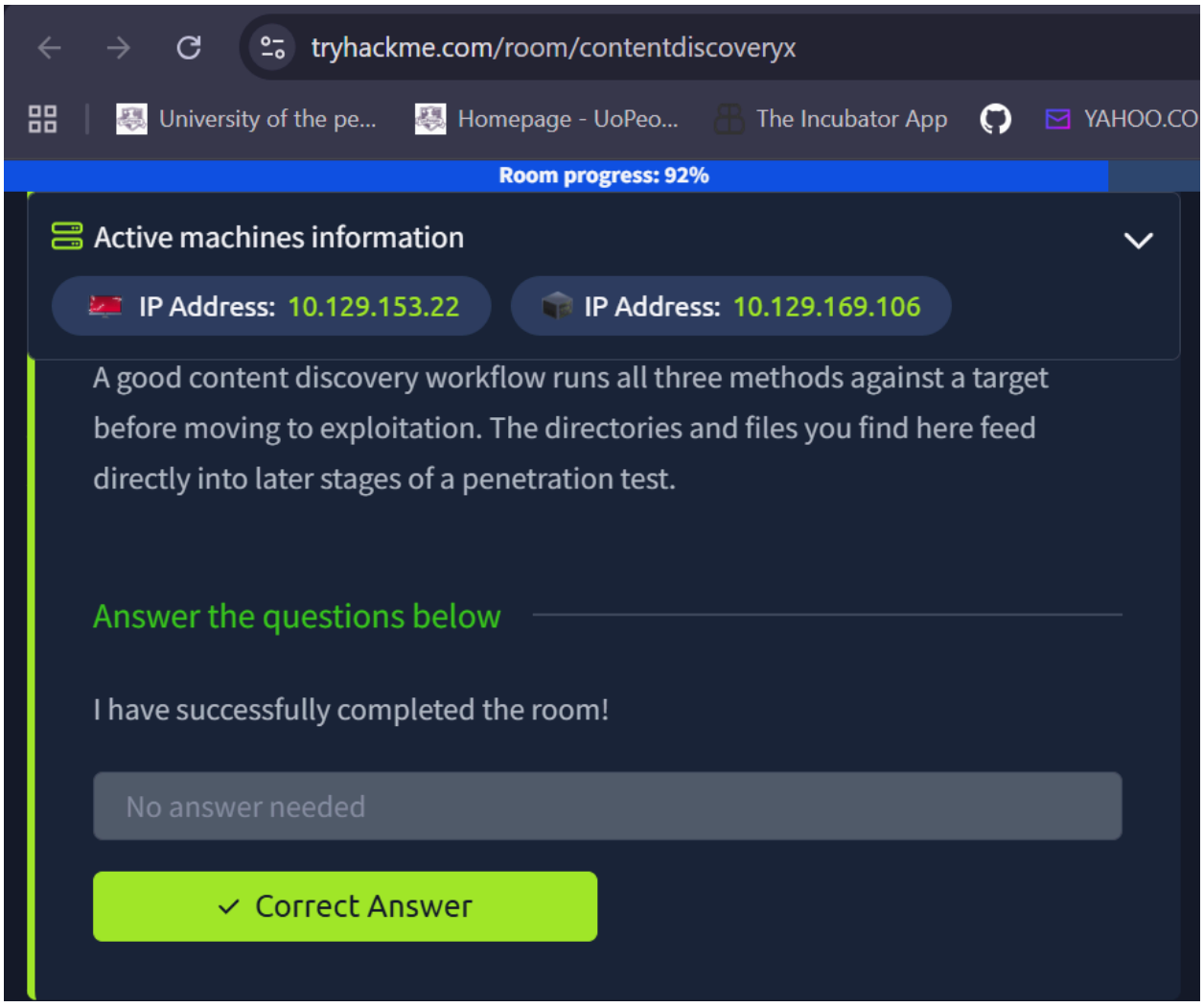
Notes:

Content discovery is one of the most important phases of web recon. The techniques work together: manual checks find quick wins, OSINT finds what the target has already made public, and automated tools cover the breadth the other two can't. A good workflow runs all three before moving to exploitation.

Methods	**Techniques**

Manual	robots.txt, sitemap.xml, favicon fingerprinting, HTTP headers, framework stack
OSINT	Google dorking, Wappalyzer, Wayback Machine, GitHub, S3 buckets
Automated	Gobuster dir, dns and vhost modes

Screenshot:



Final "What I learned" paragraph

This room taught me that a lot of a web application's attack surface is hidden rather than missing. I learned to check robots.txt, sitemap.xml, headers, and page source by hand, use OSINT such as Google dorks, Wappalyzer, the Wayback Machine, GitHub and S3 buckets, and automate discovery with Gobuster in dir, dns, and vhost modes. Identifying the framework led me to its admin portal and default credentials, which showed how small information leaks chain together. The connection problems taught me to check my network setup before blaming a command.

Completion link:

https://tryhackme.com/room/contentdiscoveryx?utm_campaign=social_share&utm_medium=social&utm_content=share-completed-room&utm_source=copy&sharerId=6a298268d8b6b164348fab72